

ogwvwpxuy

December 5, 2024

```
[ ]: from __future__ import absolute_import
from __future__ import division
from __future__ import print_function

__all__ = ['cv2_imshow', 'cv_imshow']

import cv2
from IPython import display
import PIL

def cv2_imshow(a):

    a = a.clip(0, 255).astype('uint8')
    if a.ndim == 3:
        if a.shape[2] == 4:
            a = cv2.cvtColor(a, cv2.COLOR_BGRA2RGBA)
        else:
            a = cv2.cvtColor(a, cv2.COLOR_BGR2RGB)
    display.display(PIL.Image.fromarray(a))

cv_imshow = cv2_imshow

def clear_folder(folder_path):
    """Usuń wszystkie pliki w danym folderze."""
    for filename in os.listdir(folder_path):
        file_path = os.path.join(folder_path, filename)
        try:
            if os.path.isfile(file_path):
                os.remove(file_path)
            elif os.path.isdir(file_path):
                os.rmdir(file_path)
        except Exception as e:
            print(f"Nie udało się usunąć {file_path}. Błąd: {e}")
```

```
[ ]: import cv2
import mediapipe as mp
import os
import json
import glob

# Inicjalizacja MediaPipe Pose
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5)

def extract_keypoints_from_image(image):
    """Ekstrakcja punktów kluczowych (keypoints) z obrazu"""
    results = pose.process(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    keypoints = []
    if results.pose_landmarks:
        for landmark in results.pose_landmarks.landmark:
            keypoints.append({
                "x": landmark.x,
                "y": landmark.y,
                "z": landmark.z,
                "visibility": landmark.visibility
            })
    return keypoints

def save_keypoints_to_json(keypoints, output_folder, index,
↪output_format="json"):
    """Zapisz keypoints do pliku w formacie JSON lub CSV"""
    filename = f"keypoints_{index:04d}.{output_format}" a
    output_path = os.path.join(output_folder, filename)

    if output_format == "json":
        with open(output_path, 'w') as json_file:
            json.dump(keypoints, json_file, indent=4)
            print(f"Keypoints zapisane do: {output_path}")

    elif output_format == "csv":
        with open(output_path, 'w', newline='') as csv_file:
            writer = csv.writer(csv_file)
            for point in keypoints:
                writer.writerow(point)
            print(f"Keypoints zapisane do: {output_path}")

def process_image_and_save_keypoints(image_path, output_folder, index,
↪output_format="json"):
    """Wczytaj obraz, ekstraktuj keypoints i zapisz do pliku JSON lub CSV"""
```

```

image = cv2.imread(image_path)

if image is None:
    print(f"Nie udało się wczytać obrazu z {image_path}")
    return

keypoints = extract_keypoints_from_image(image)

save_keypoints_to_json(keypoints, output_folder, index, output_format)

def load_images_from_folder(folder_path):
    """Wczytaj obrazy z folderu"""
    supported_formats = ['*.jpg', '*.png']
    image_files = []
    for fmt in supported_formats:
        image_files.extend(glob.glob(os.path.join(folder_path, fmt)))
    image_files.sort()
    return image_files

def process_folder_and_save_keypoints(images_folder, output_folder,
    ↪output_format="json"):
    """Przetwórz wszystkie obrazy w folderze i zapisz keypoints do plików JSON,
    ↪lub CSV"""
    image_files = load_images_from_folder(images_folder)

    for idx, image_path in enumerate(image_files, start=1):
        process_image_and_save_keypoints(image_path, output_folder, idx,
    ↪output_format)

class KeypointDataLoader:
    def __init__(self, data_path, file_format="json"):
        self.data_path = data_path
        self.file_format = file_format
        self.keypoints = self.load_keypoints()

    def load_keypoints(self):
        """Wczytaj keypoints z pliku JSON"""
        keypoints = []
        if self.file_format == "json":
            with open(self.data_path, 'r') as jsonfile:
                keypoints = json.load(jsonfile)
        return keypoints

    def visualize_keypoints_on_image(self, image):
        """Wizualizuj keypoints na obrazie"""

```

```

        for keypoint in self.keypoints:
            x, y = int(keypoint["x"] * image.shape[1]), int(keypoint["y"] *
↳image.shape[0])
            cv2.circle(image, (x, y), 5, (0, 255, 0), -1)
        return image

def main():
    # Ścieżki do folderów
    images_folder = "/content/images" # Folder z obrazami
    output_folder = "/content/keypoints" # Folder do zapisu plików JSON

    os.makedirs(output_folder, exist_ok=True)
    clear_folder(output_folder)
    process_folder_and_save_keypoints(images_folder, output_folder,
↳output_format="json")

    image_files = load_images_from_folder(images_folder)
    json_files = sorted(glob.glob(os.path.join(output_folder, "*.json")))

    for image_path, json_path in zip(image_files, json_files):
        loader = KeypointDataLoader(json_path, file_format="json")

        # Wczytaj obraz i wizualizuj keypoints
        image = cv2.imread(image_path)
        loader = KeypointDataLoader(json_path, file_format="json")
        img_with_keypoints = loader.visualize_keypoints_on_image(image)
        cv2.imshow(img_with_keypoints)

if __name__ == '__main__':
    main()

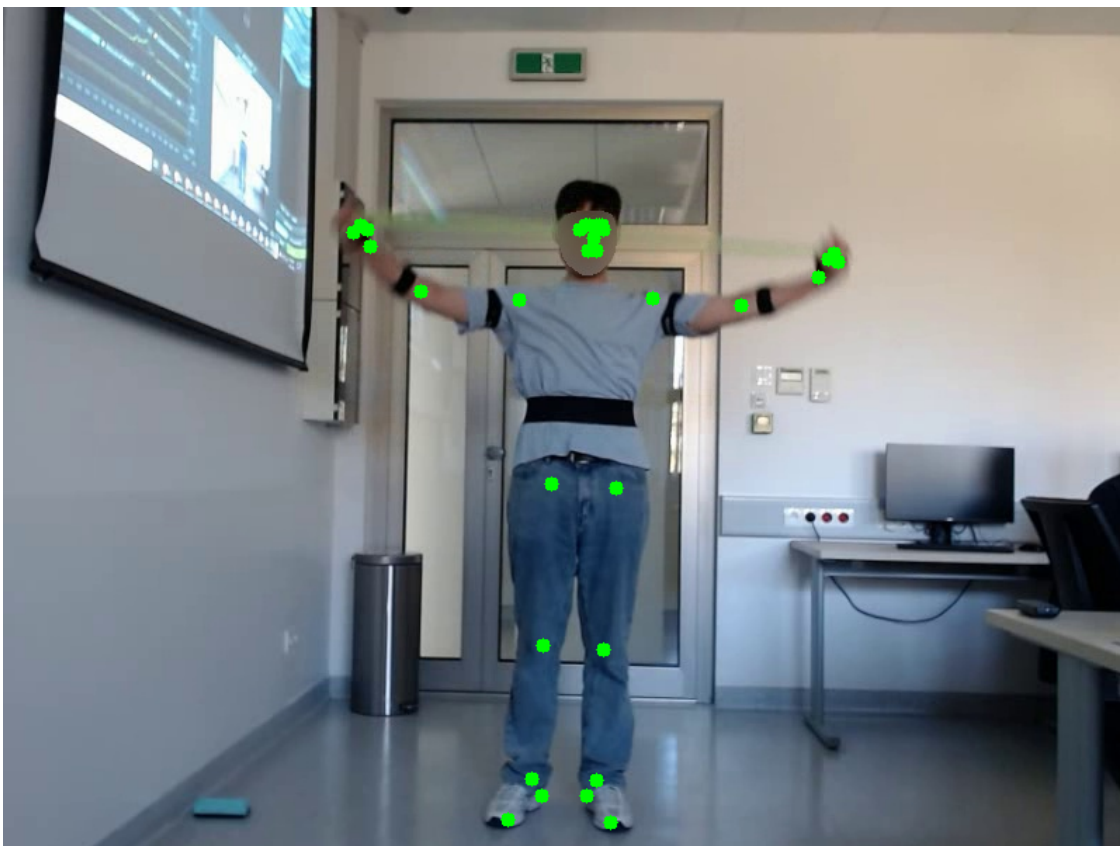
```

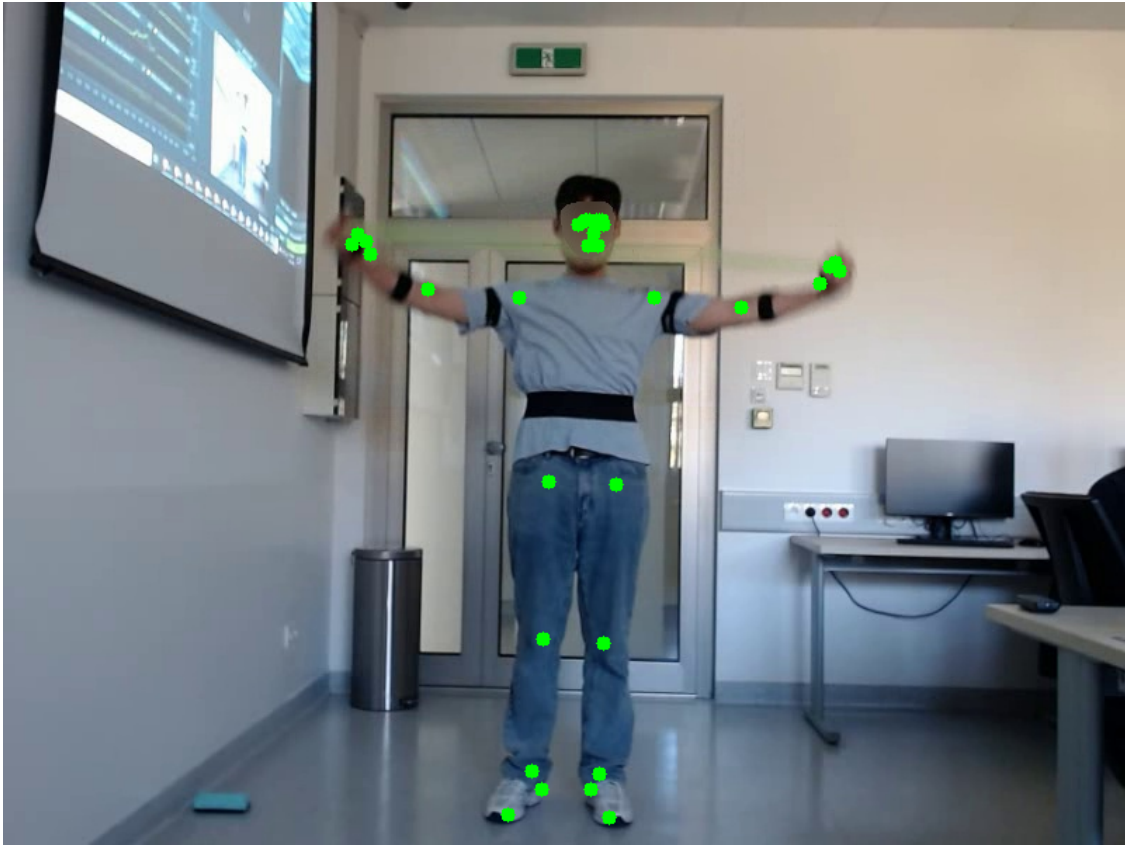
```

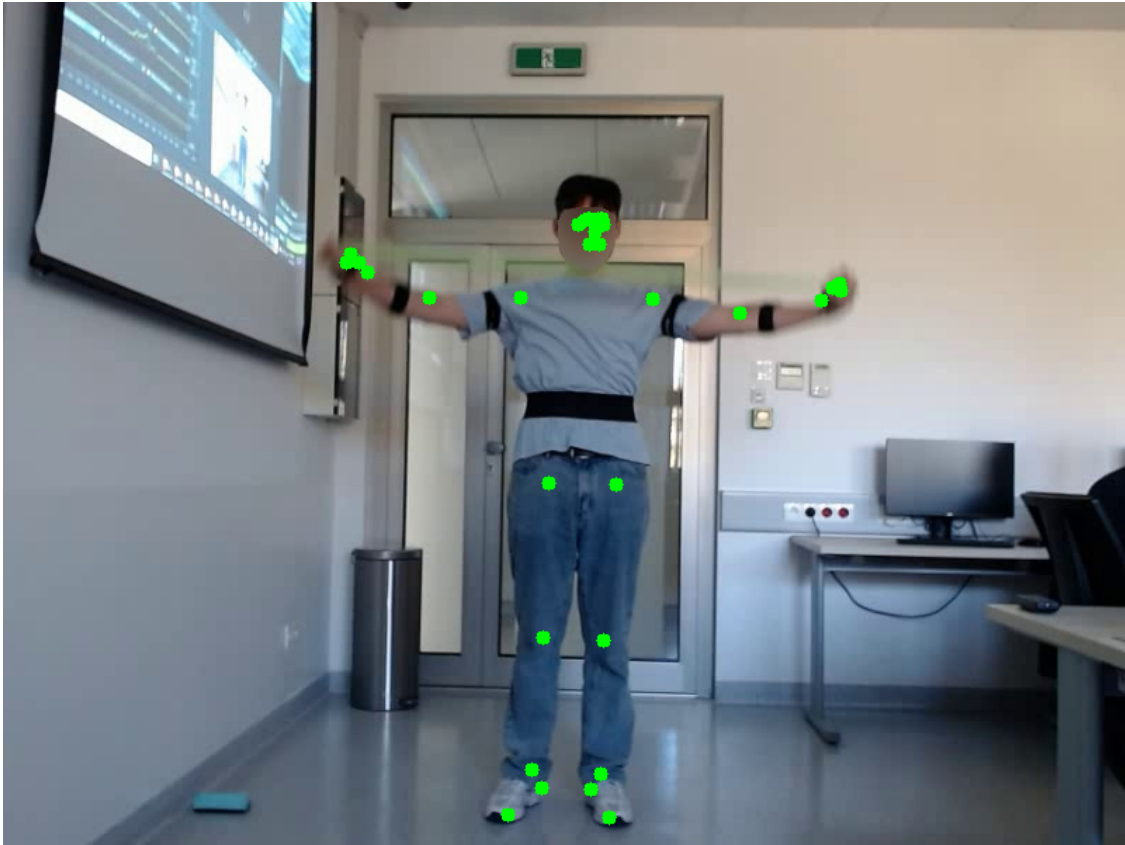
Keypoints zapisane do: /content/keypoints/keypoints_0001.json
Keypoints zapisane do: /content/keypoints/keypoints_0002.json
Keypoints zapisane do: /content/keypoints/keypoints_0003.json
Keypoints zapisane do: /content/keypoints/keypoints_0004.json
Keypoints zapisane do: /content/keypoints/keypoints_0005.json
Keypoints zapisane do: /content/keypoints/keypoints_0006.json
Keypoints zapisane do: /content/keypoints/keypoints_0007.json
Keypoints zapisane do: /content/keypoints/keypoints_0008.json
Keypoints zapisane do: /content/keypoints/keypoints_0009.json
Keypoints zapisane do: /content/keypoints/keypoints_0010.json

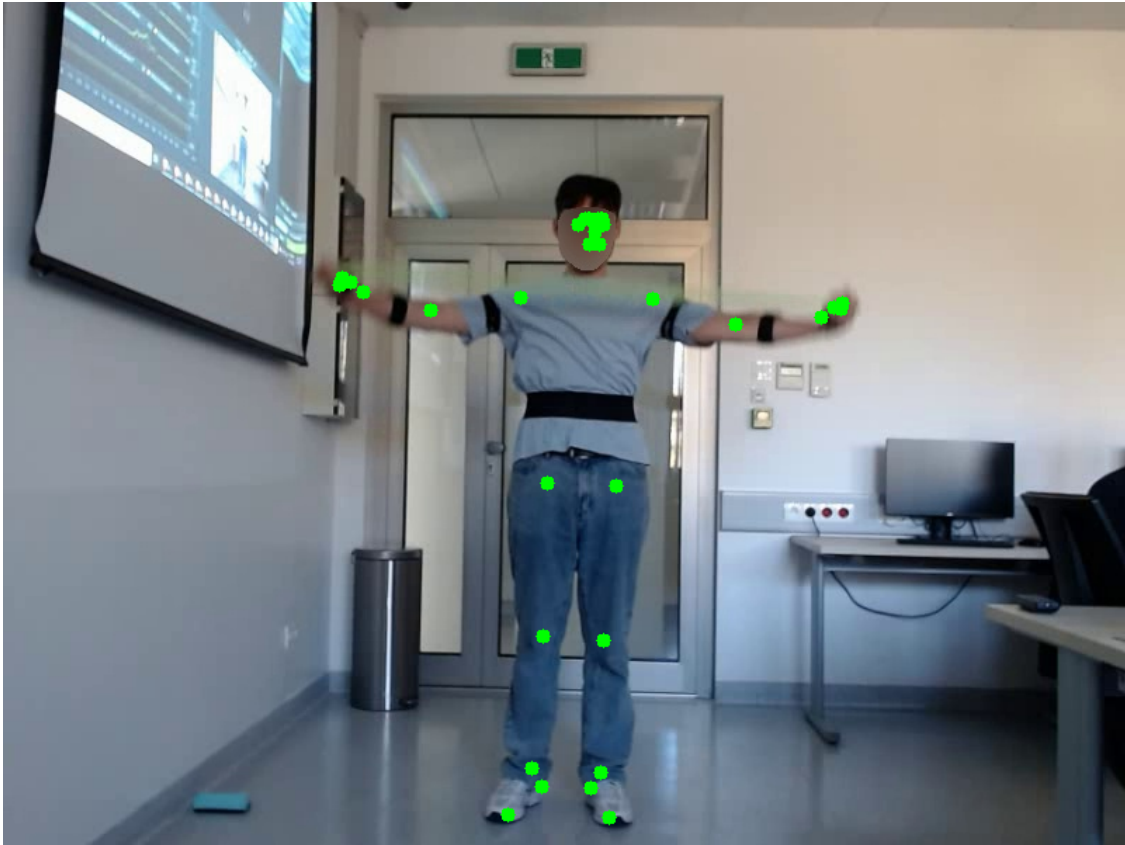
```

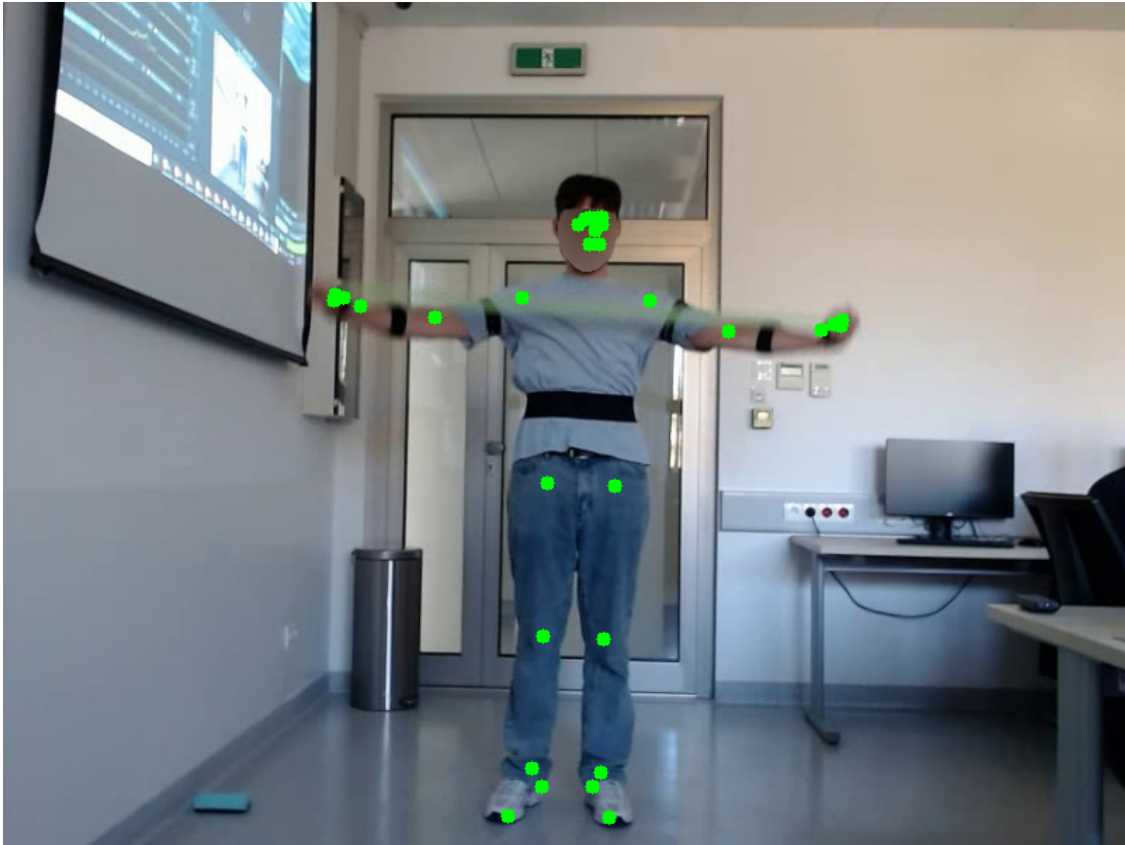

Keypoints zapisane do: /content/keypoints/keypoints_0011.json
Keypoints zapisane do: /content/keypoints/keypoints_0012.json
Keypoints zapisane do: /content/keypoints/keypoints_0013.json
Keypoints zapisane do: /content/keypoints/keypoints_0014.json
Keypoints zapisane do: /content/keypoints/keypoints_0015.json
Keypoints zapisane do: /content/keypoints/keypoints_0016.json
Keypoints zapisane do: /content/keypoints/keypoints_0017.json
Keypoints zapisane do: /content/keypoints/keypoints_0018.json
Keypoints zapisane do: /content/keypoints/keypoints_0019.json
Keypoints zapisane do: /content/keypoints/keypoints_0020.json
Keypoints zapisane do: /content/keypoints/keypoints_0021.json
Keypoints zapisane do: /content/keypoints/keypoints_0022.json
Keypoints zapisane do: /content/keypoints/keypoints_0023.json
Keypoints zapisane do: /content/keypoints/keypoints_0024.json
Keypoints zapisane do: /content/keypoints/keypoints_0025.json
Keypoints zapisane do: /content/keypoints/keypoints_0026.json
Keypoints zapisane do: /content/keypoints/keypoints_0027.json
Keypoints zapisane do: /content/keypoints/keypoints_0028.json

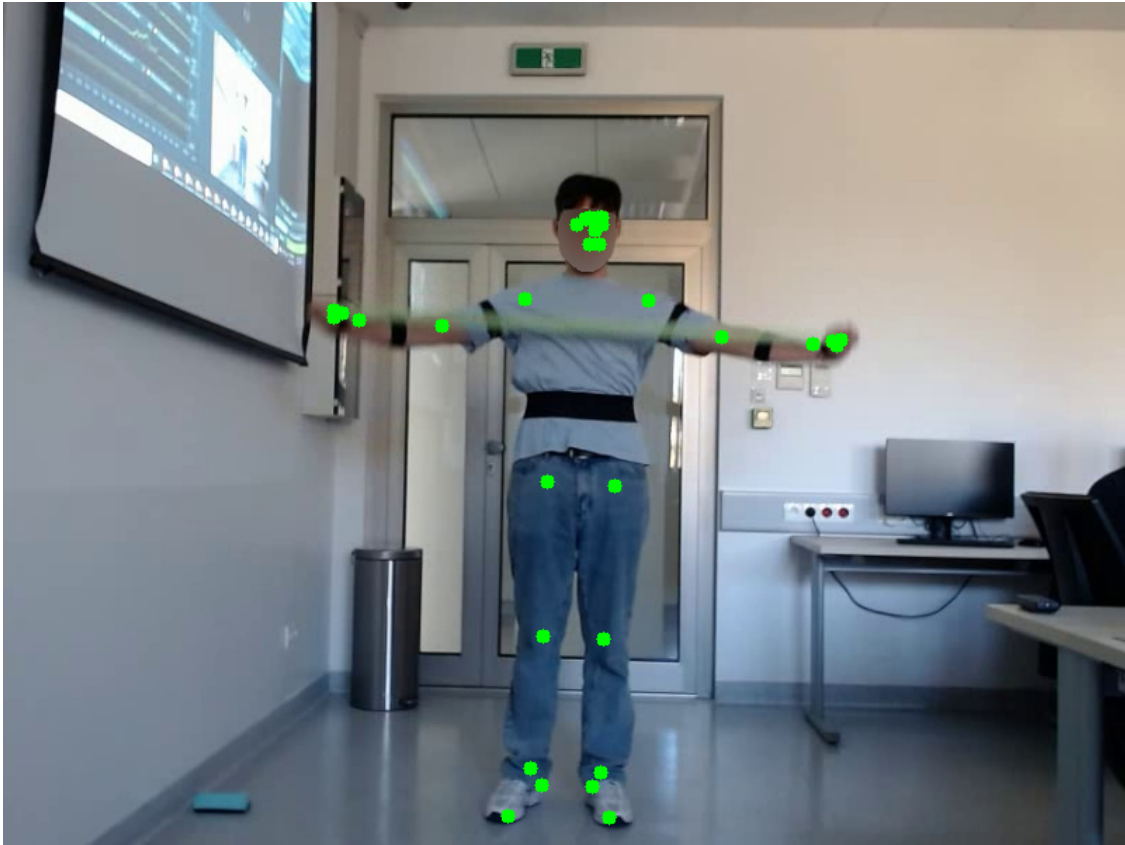


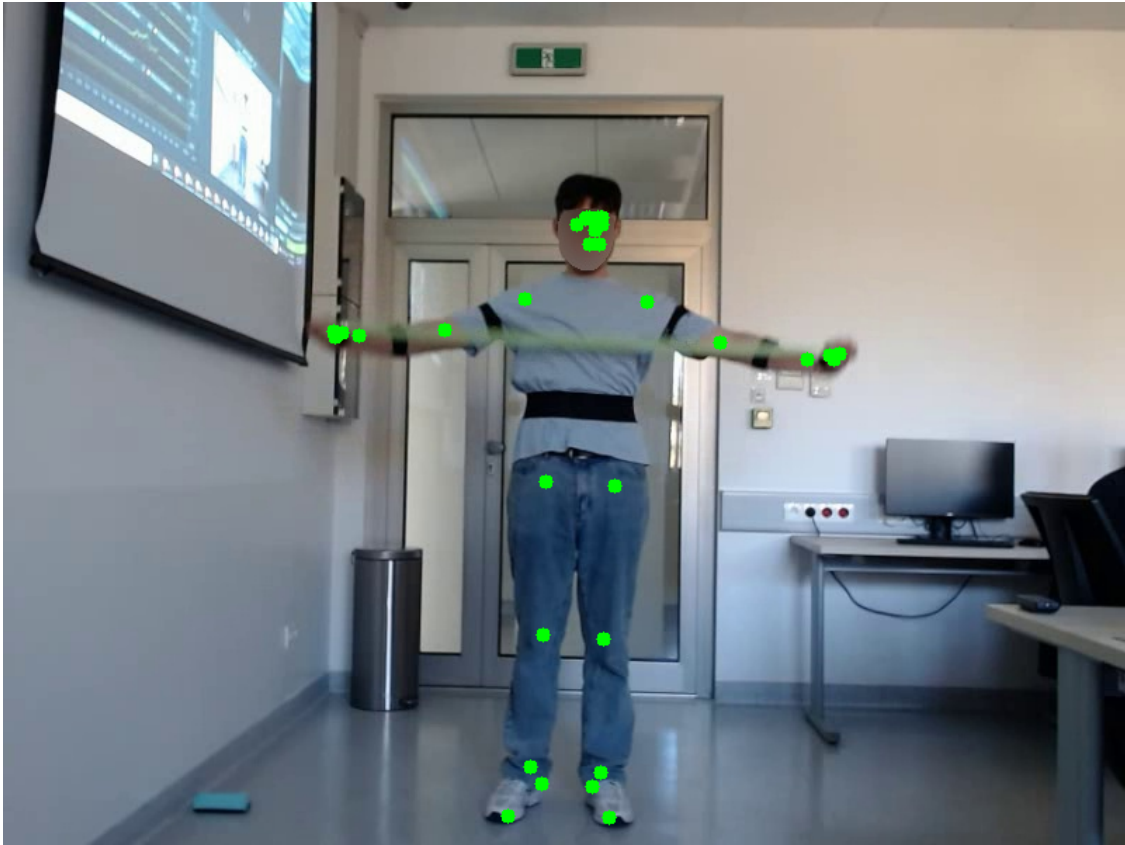


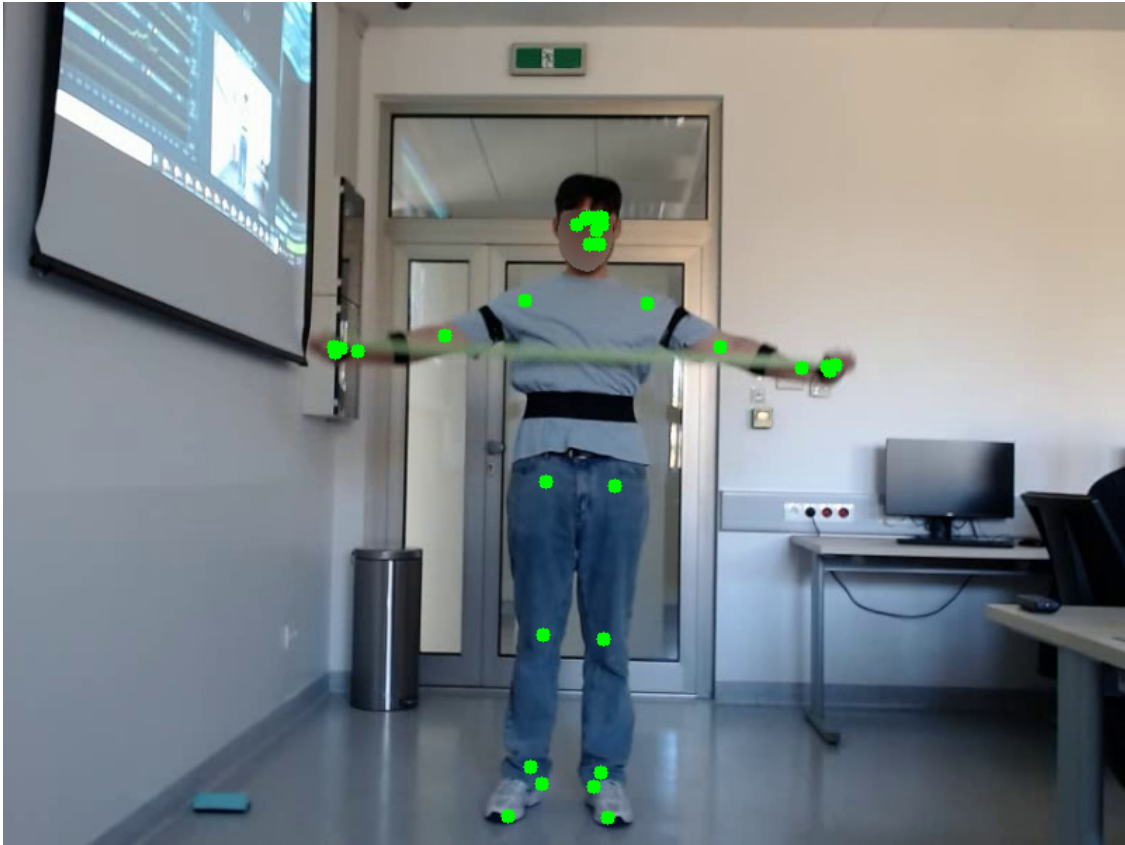


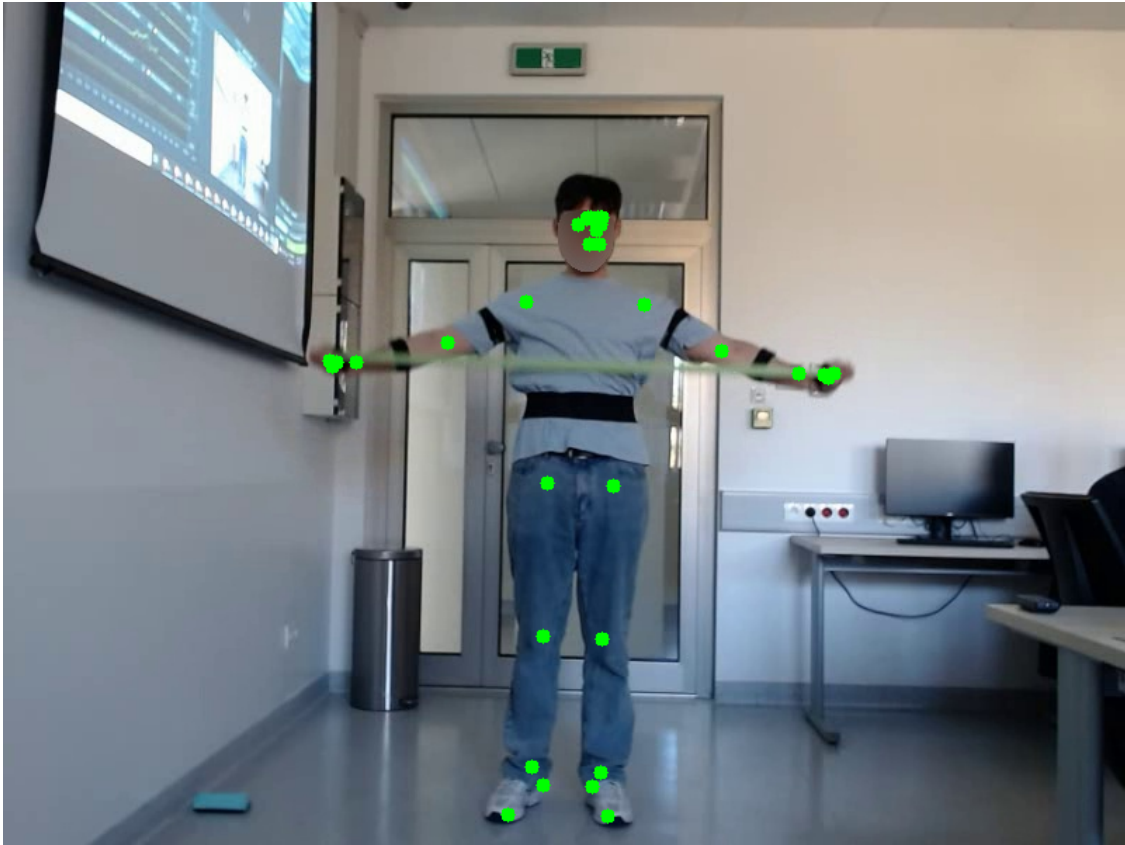


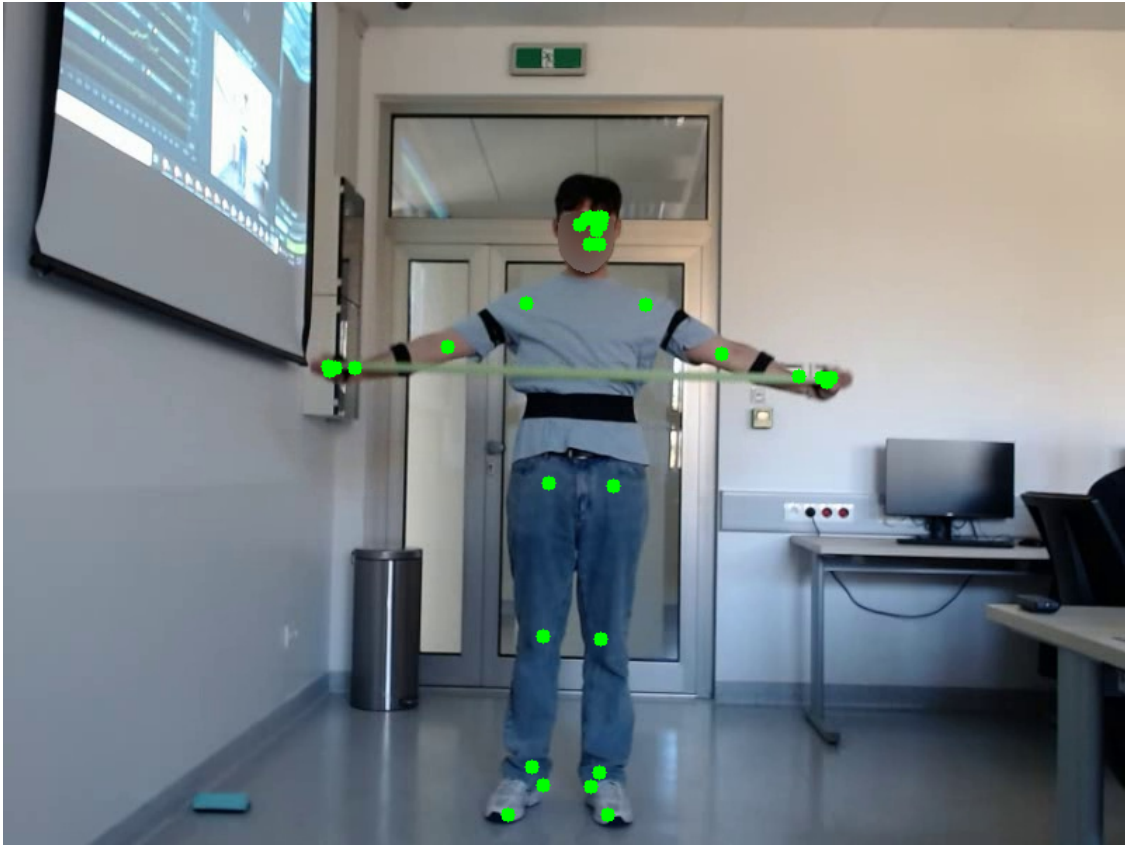


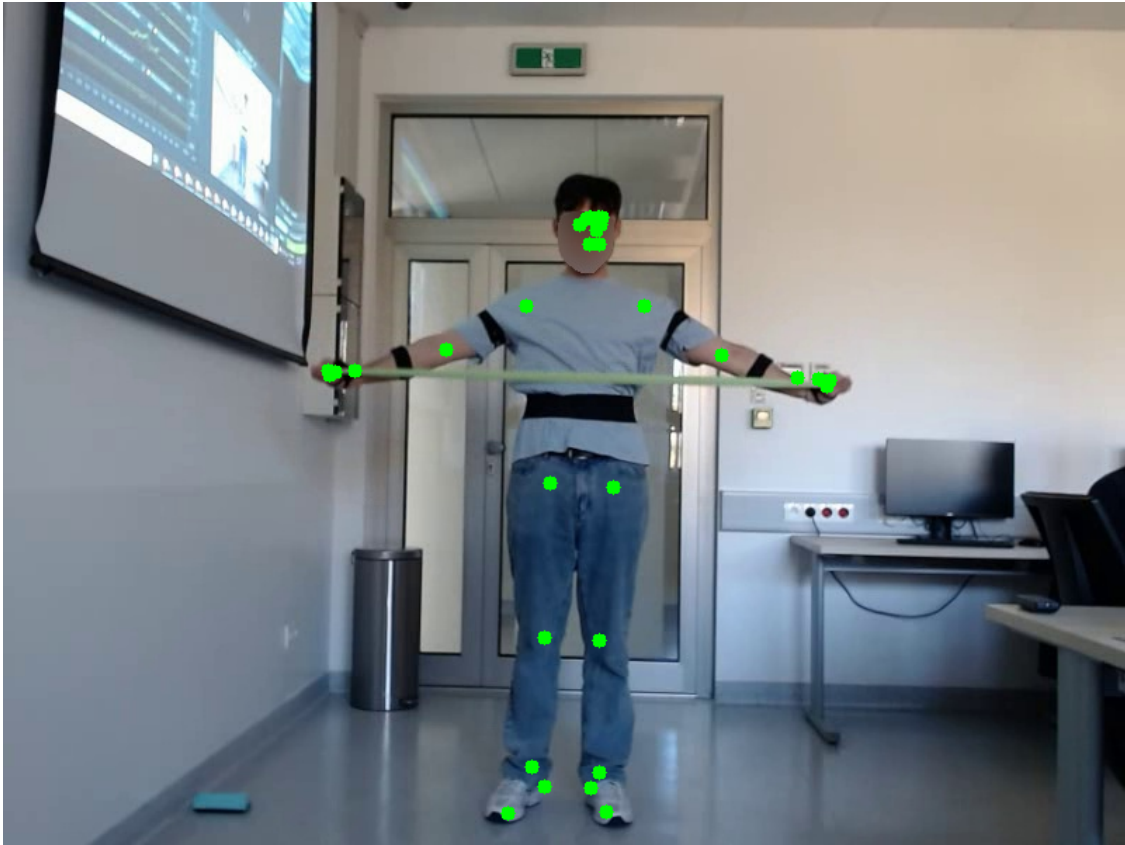


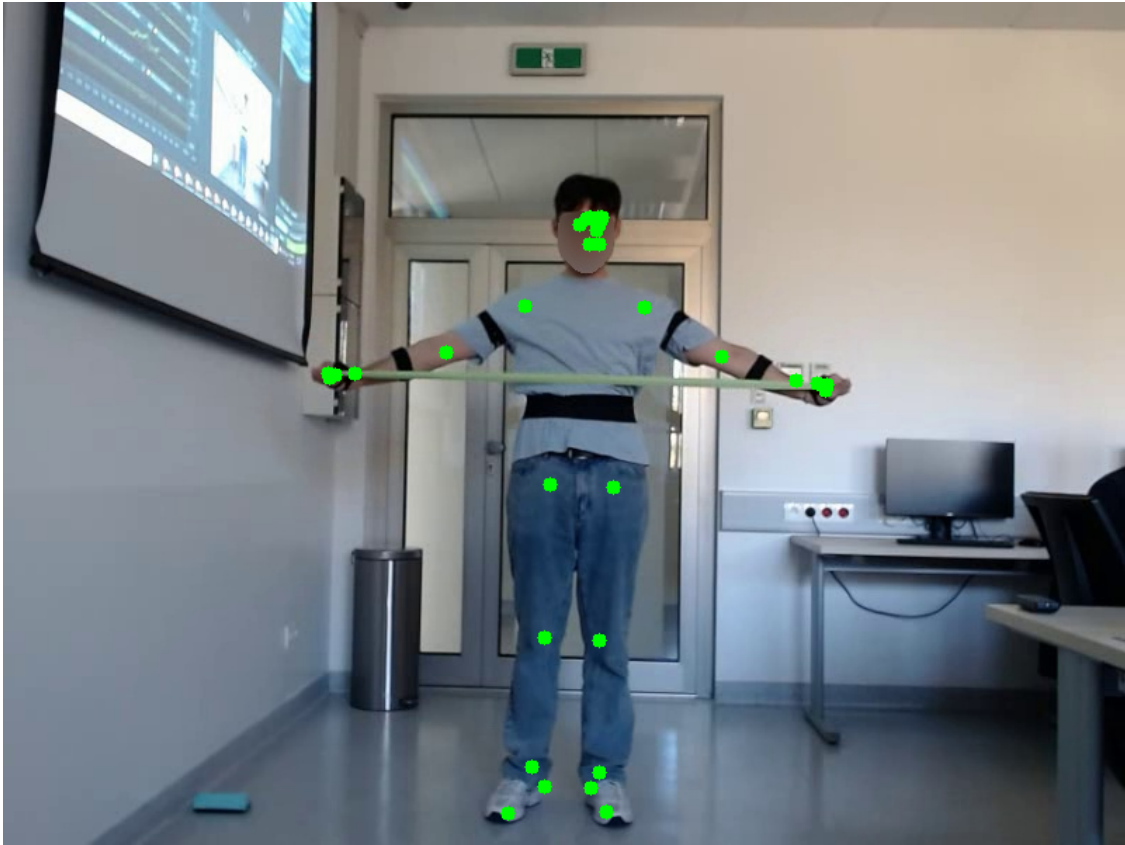


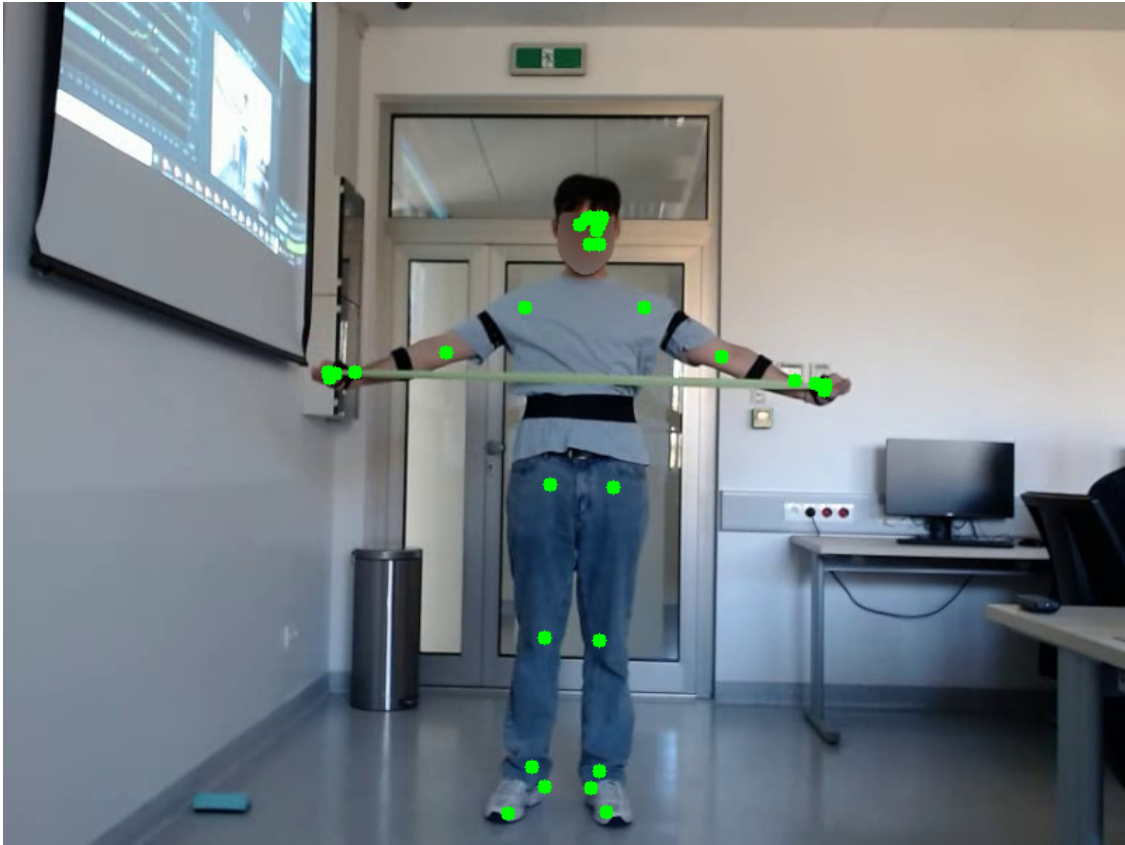


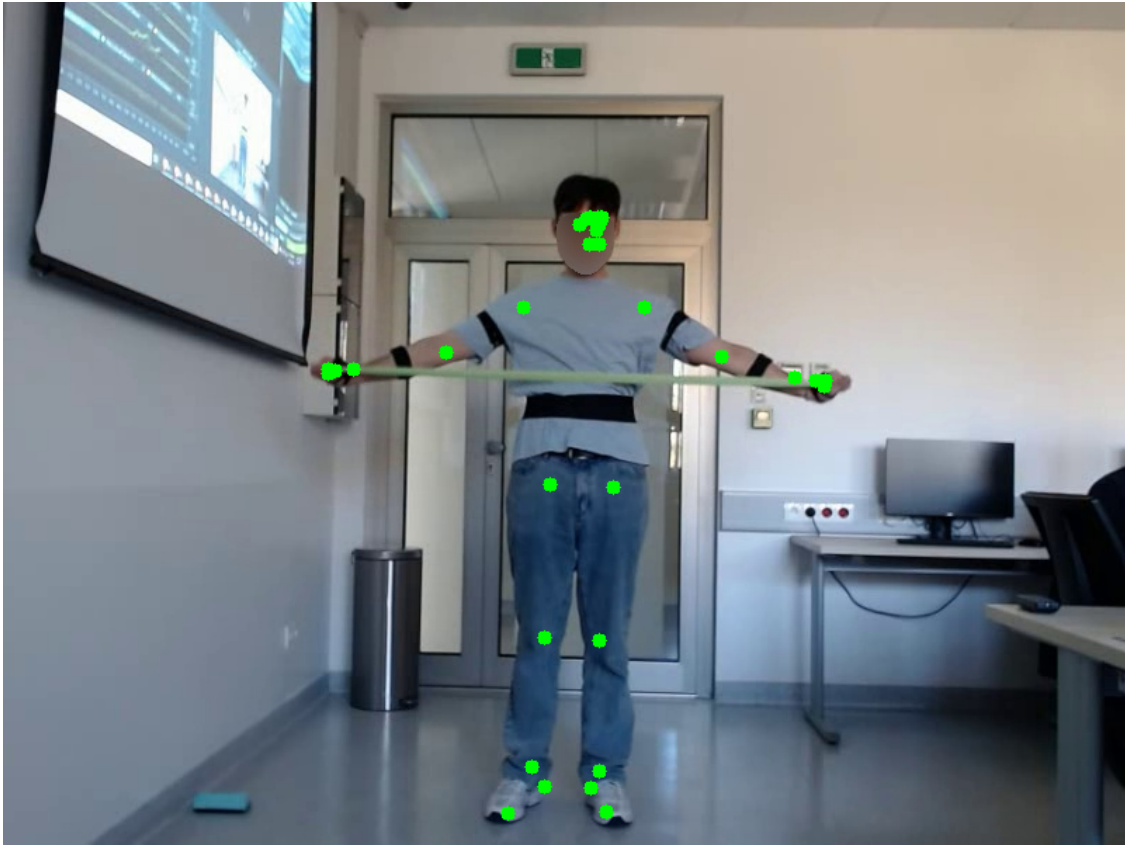


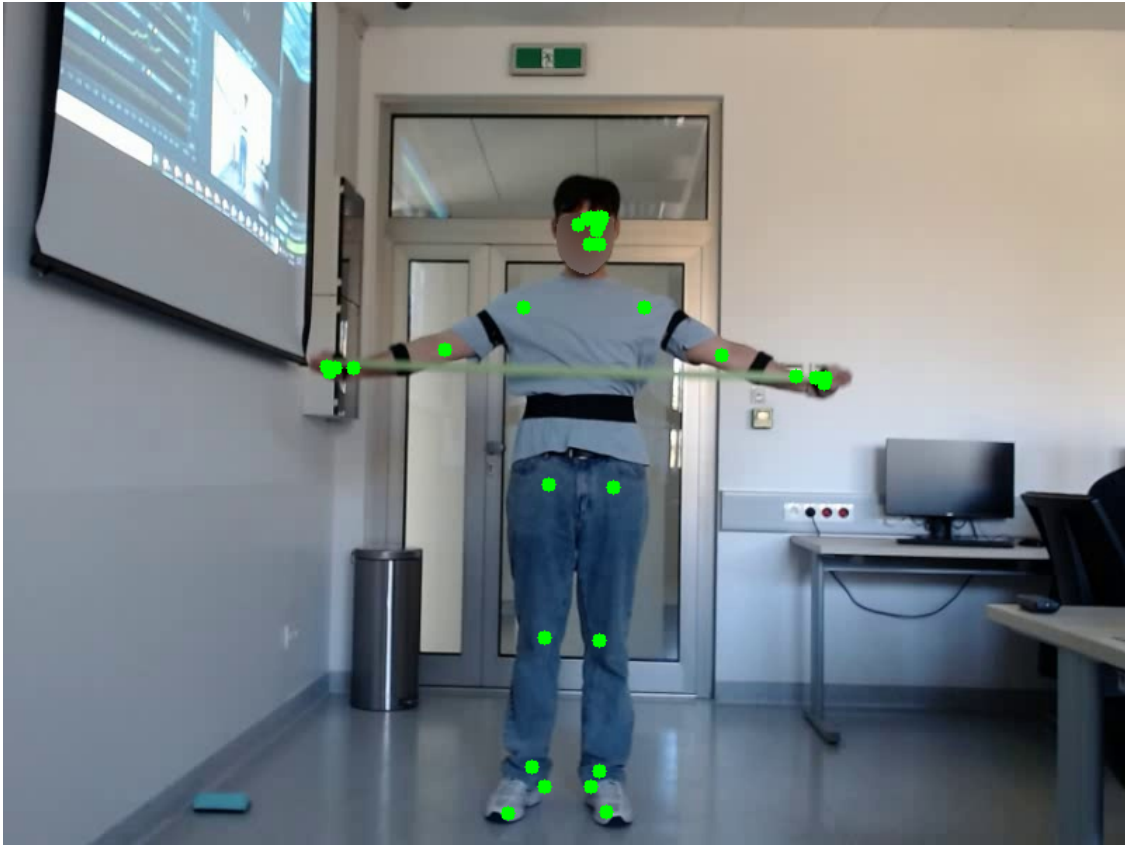


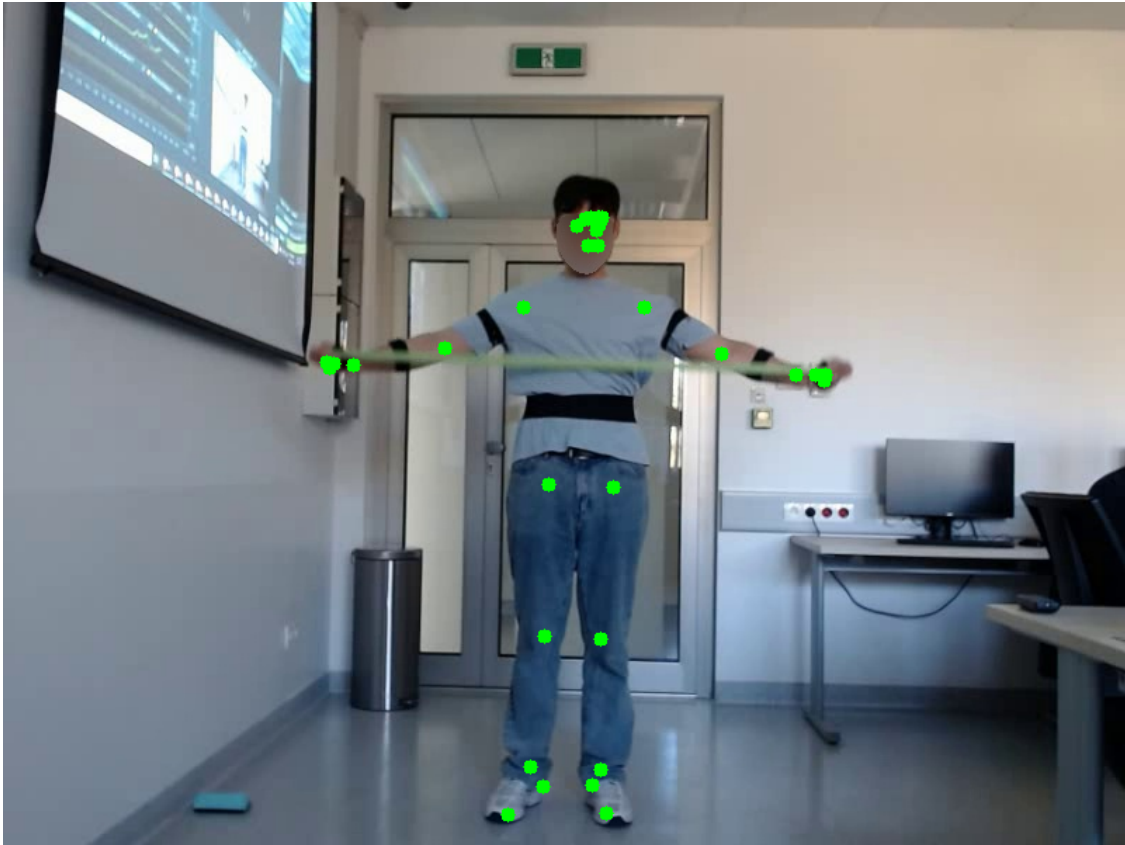


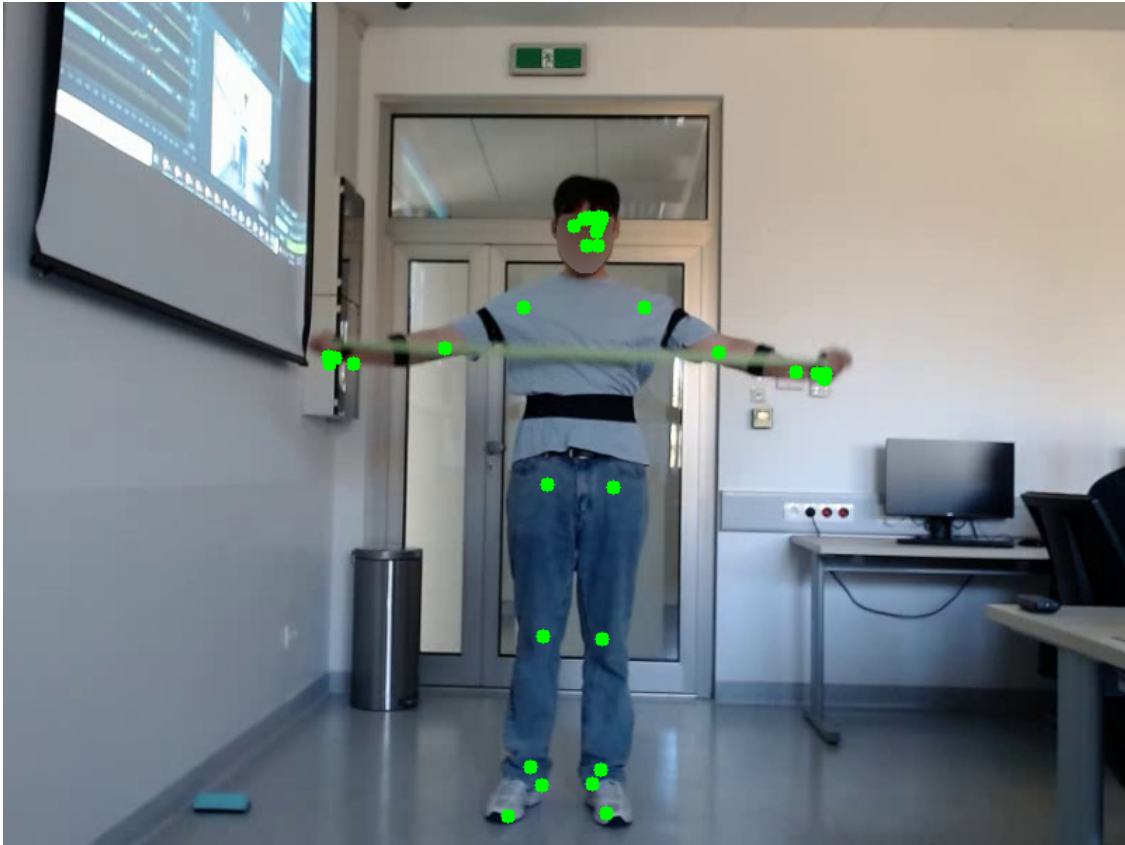


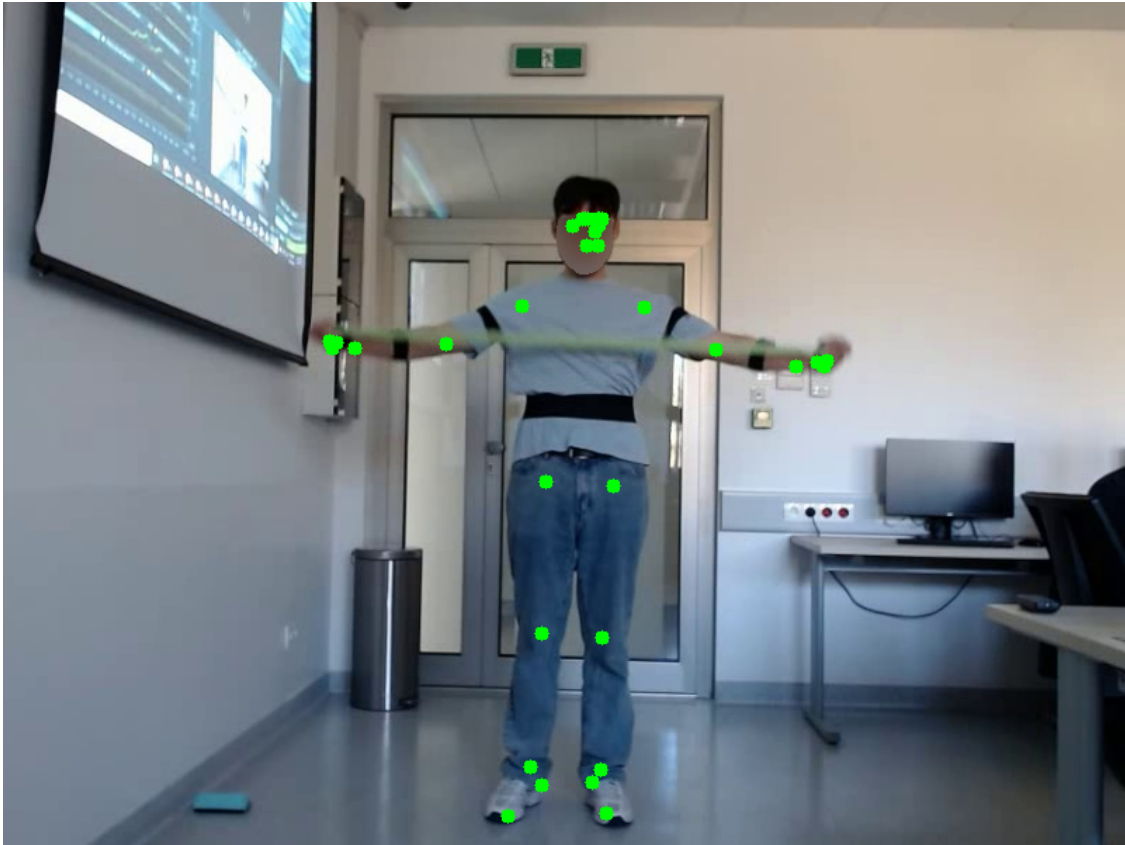


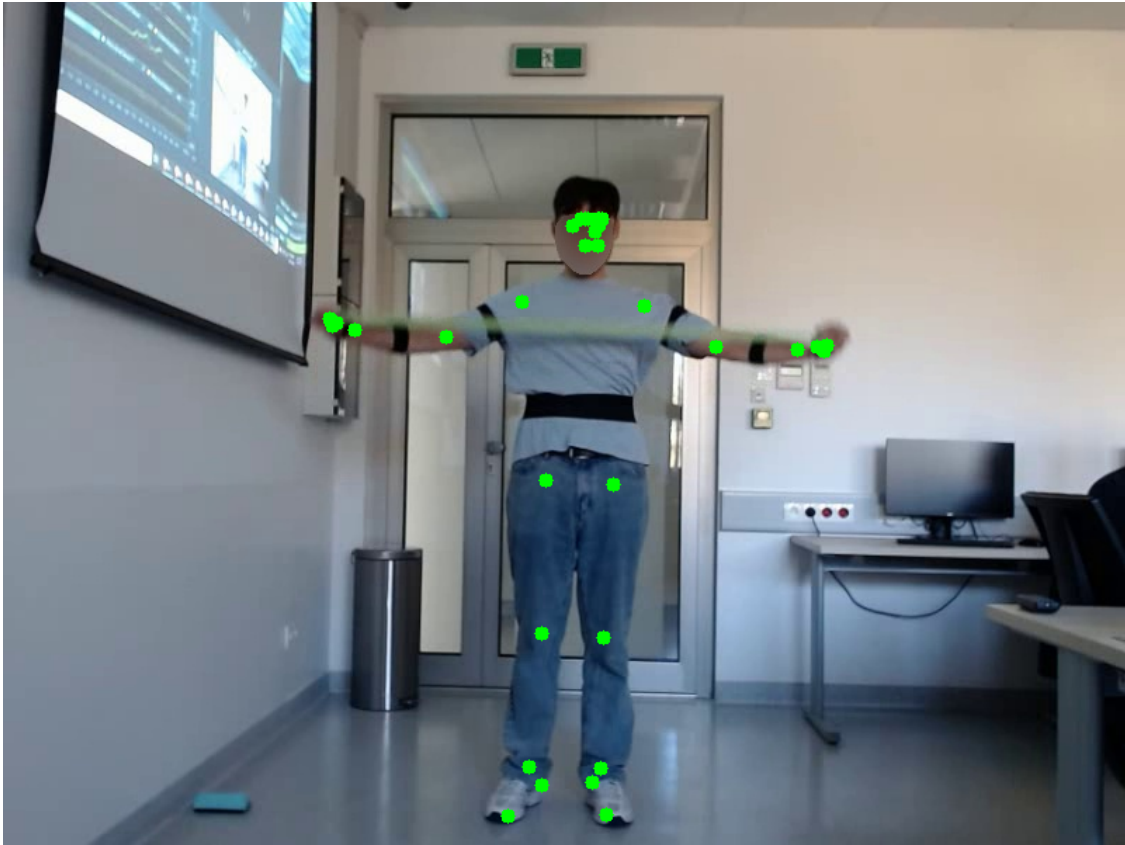


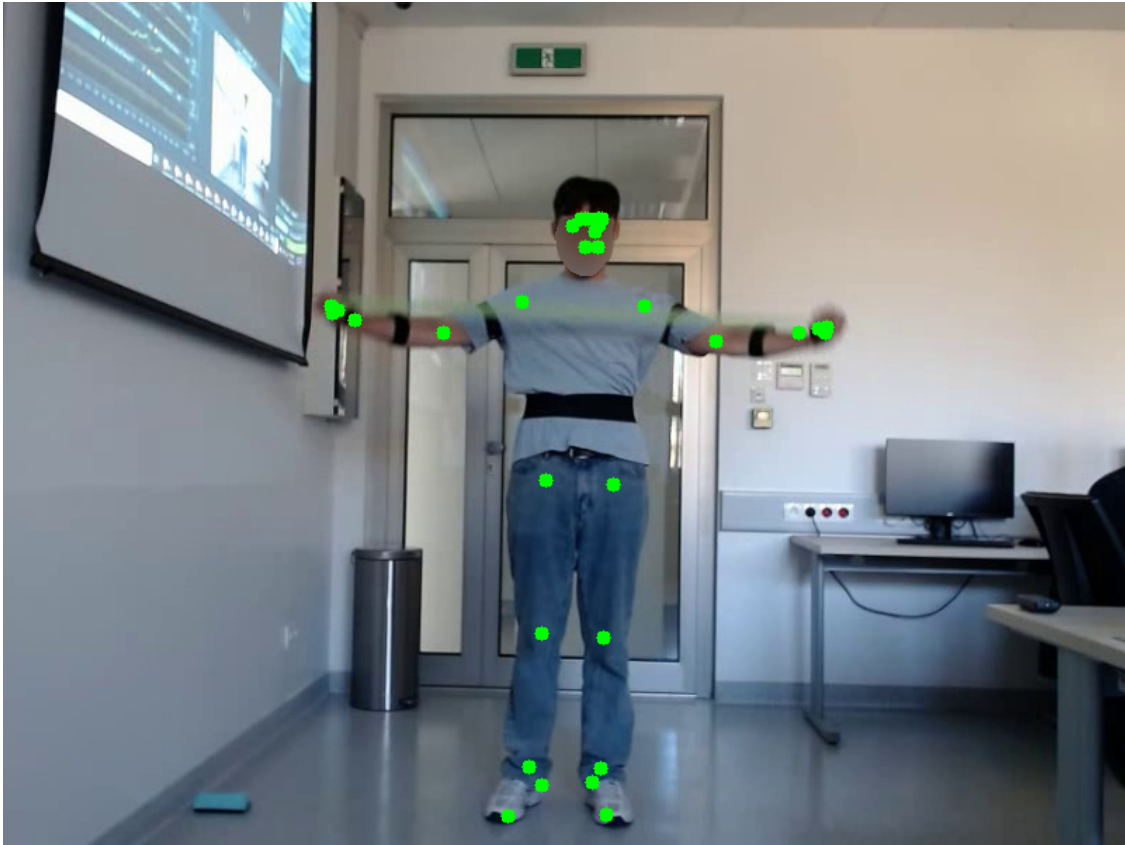


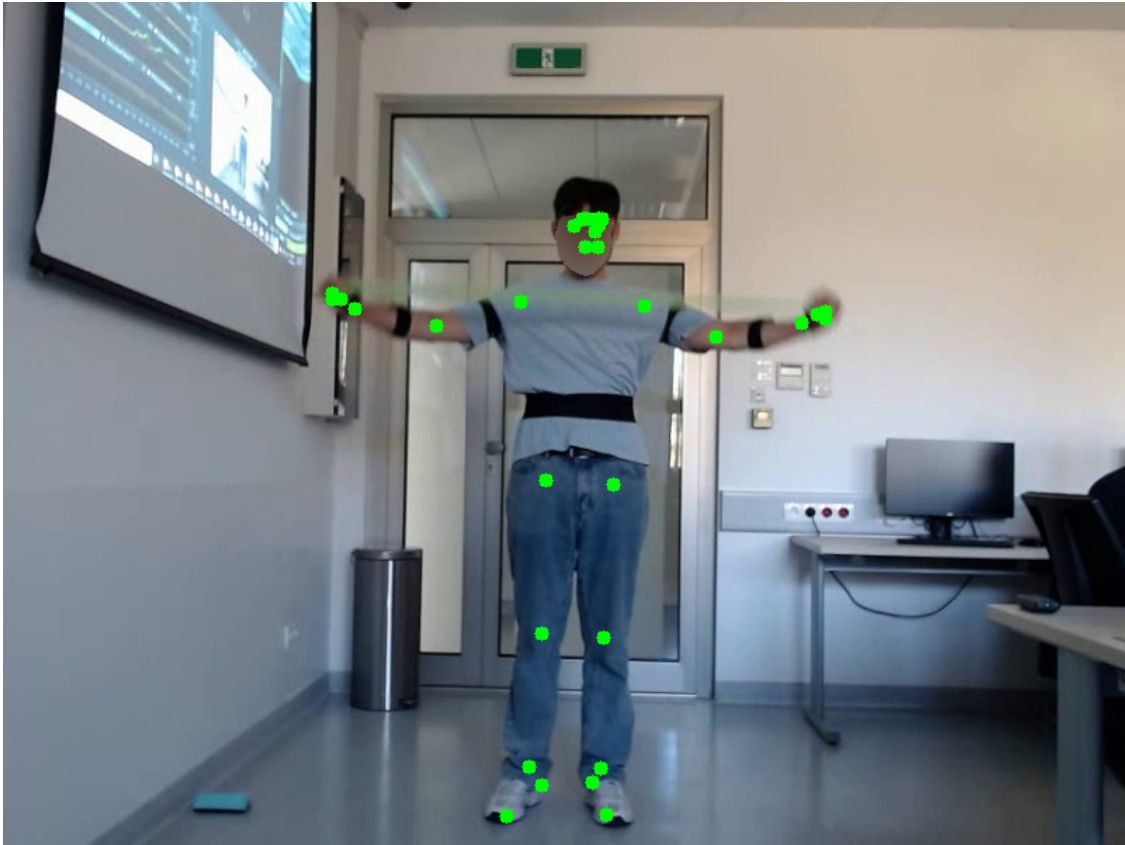


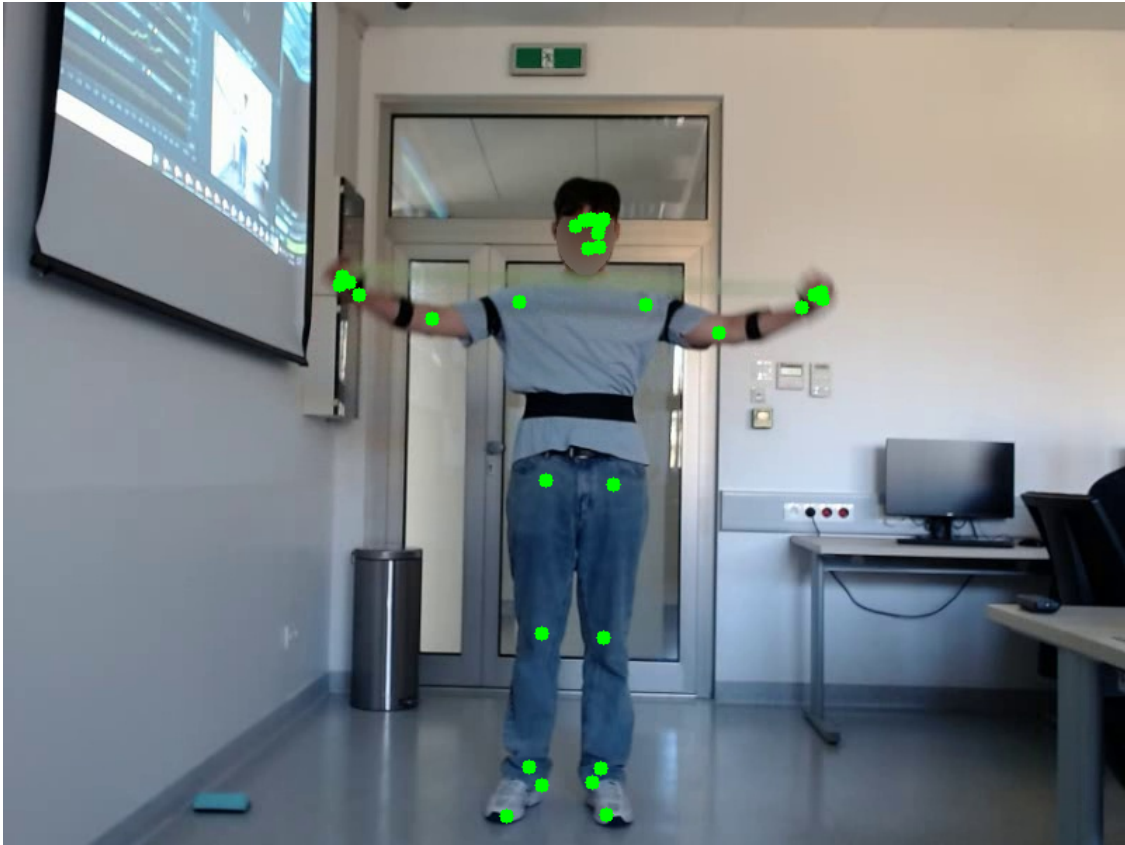


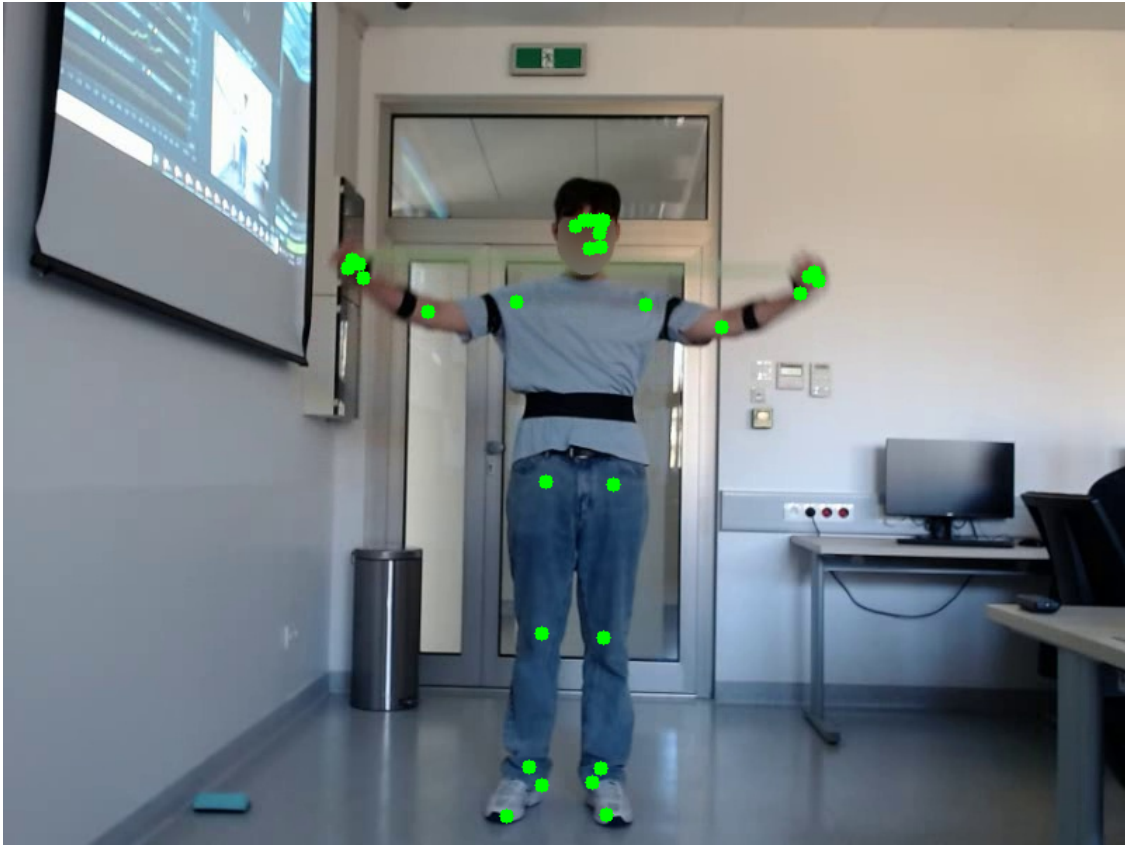


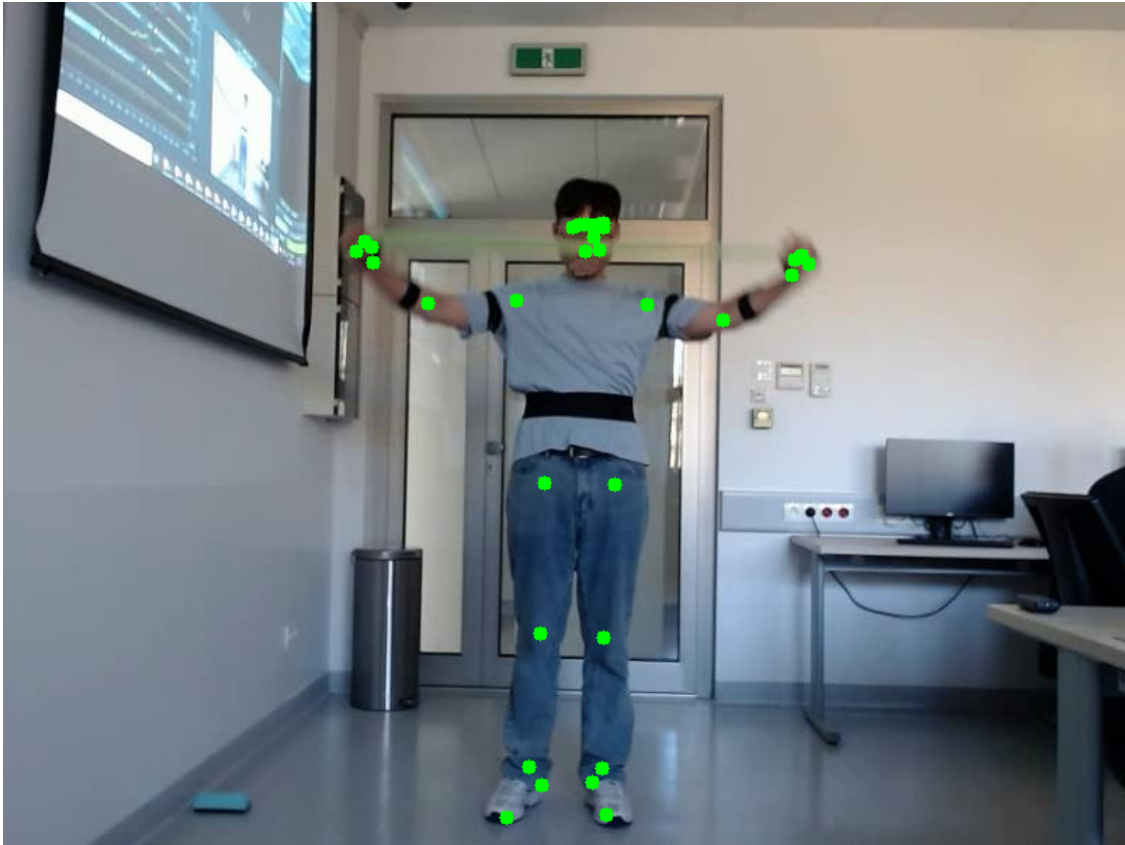


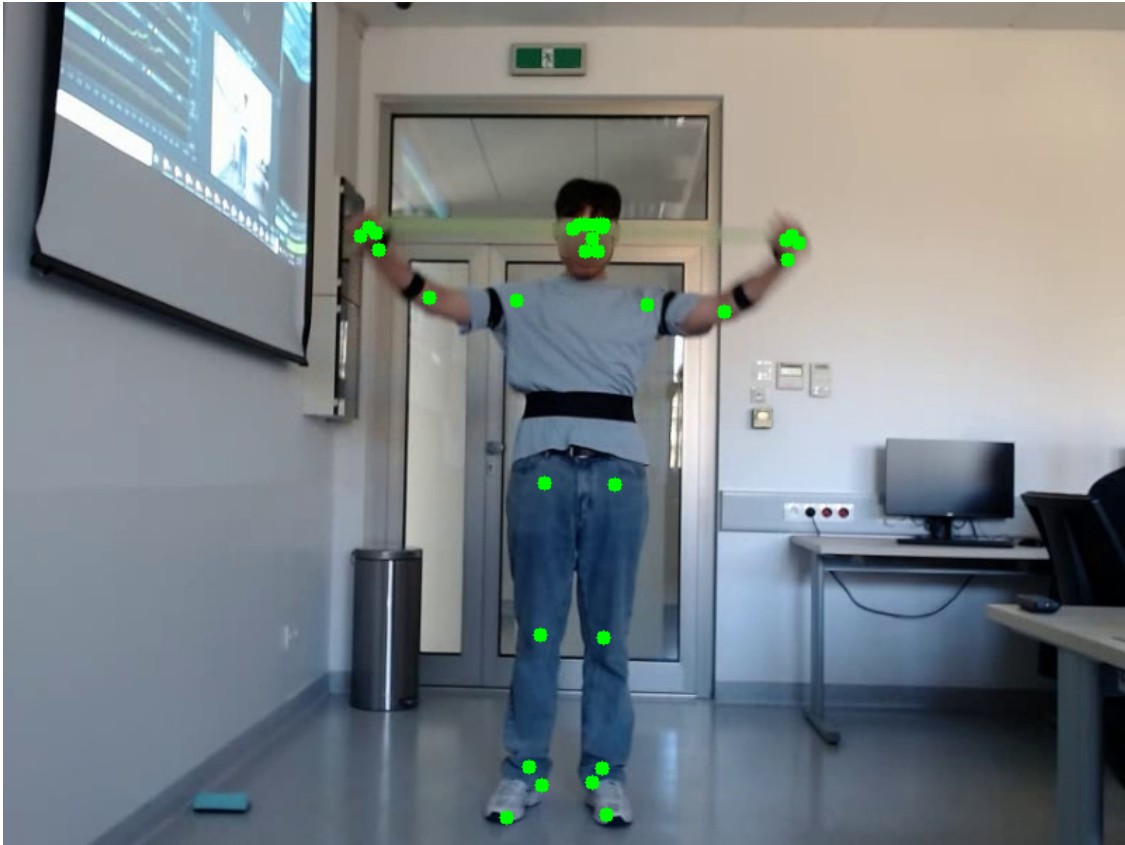


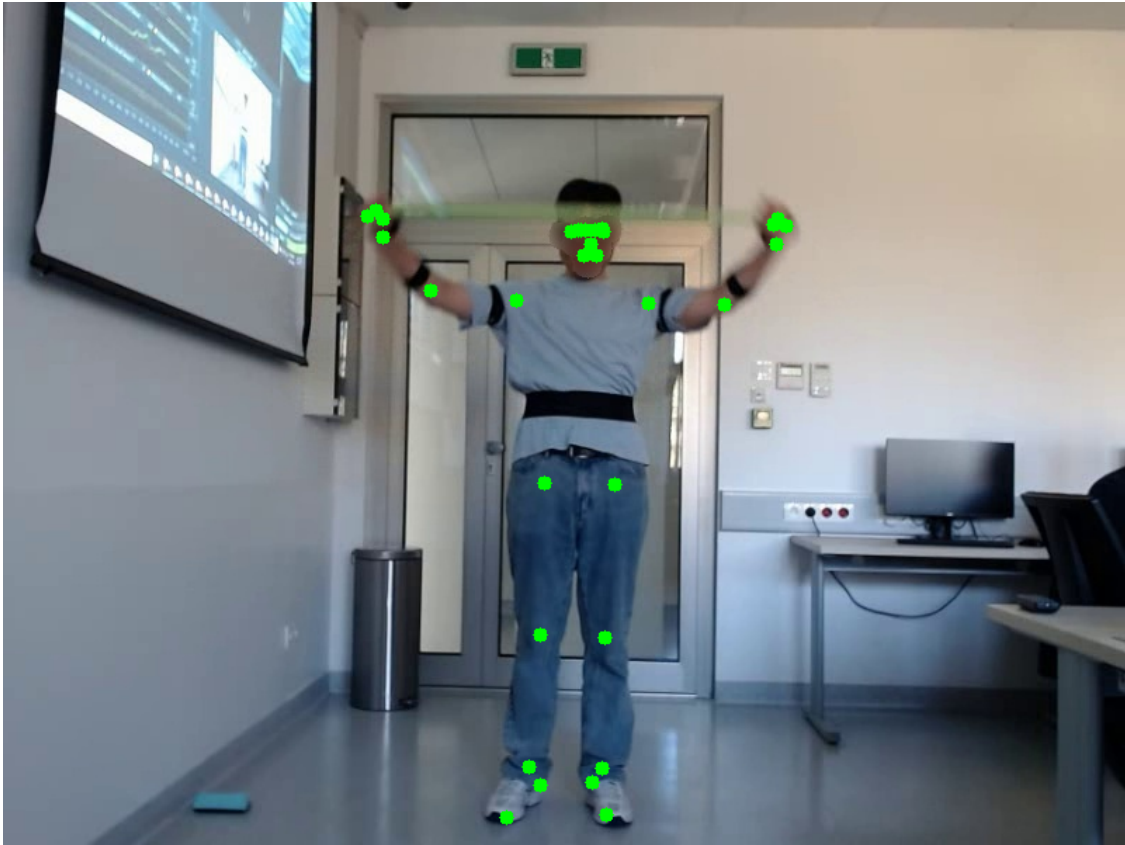


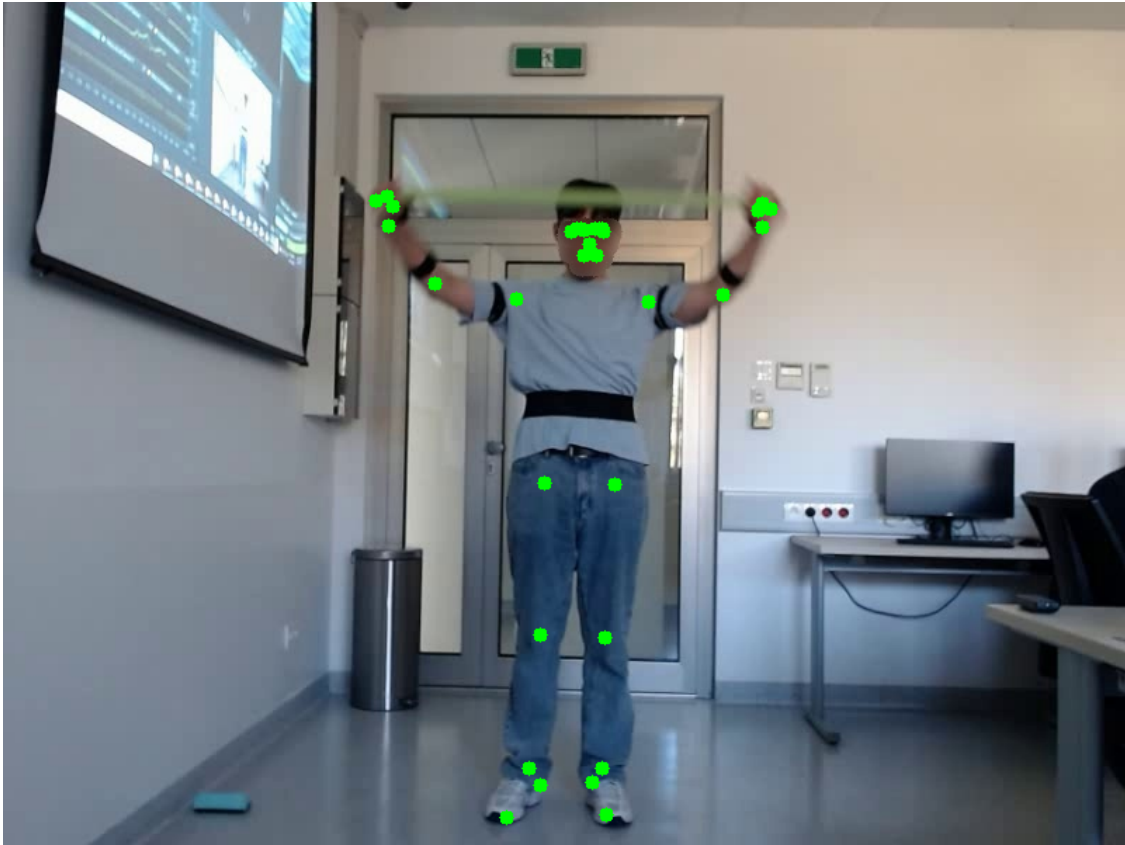


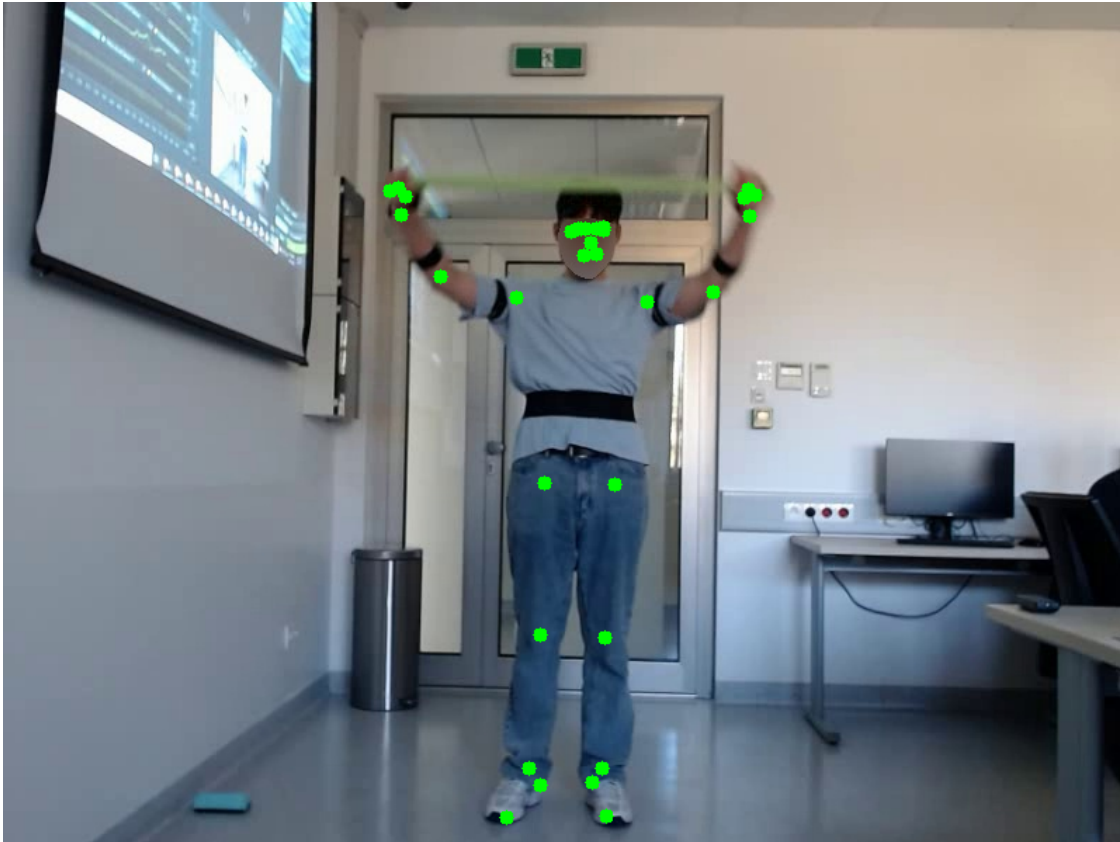












```
[ ]: import random

def augment_keypoints(keypoints, max_shift=10, scale_range=(0.8, 1.2)):
    """Augmentacja danych szkieletowych: przesunięcie i skalowanie (z
    uwzględnieniem struktury z 'x', 'y', 'z', 'visibility')"""

    shift_x = random.uniform(-max_shift, max_shift)
    shift_y = random.uniform(-max_shift, max_shift)

    # Skalowanie punktów w przestrzeni 2D
    scale = random.uniform(scale_range[0], scale_range[1])

    augmented_keypoints = []

    for point in keypoints:
        x = point['x']
        y = point['y']
        z = point['z']
        visibility = point['visibility']

        scale = random.uniform(-0.08, 0.08)
```

```

        augmented_x = (x * (1 + scale))
        augmented_y = (y * (1 + scale))
        augmented_z = z * (1 + scale)

        augmented_keypoints.append({
            'x': augmented_x,
            'y': augmented_y,
            'z': augmented_z,
            'visibility': visibility
        })

    return augmented_keypoints

def load_keypoints_from_json(json_path):
    """Wczytaj keypoints z pliku JSON"""
    with open(json_path, 'r') as file:
        keypoints = json.load(file)
    return keypoints

def save_keypoints_to_json(keypoints, output_folder, filename):
    """Zapisz zaugmentowane keypoints do pliku JSON"""
    output_path = os.path.join(output_folder, filename)
    with open(output_path, 'w') as json_file:
        json.dump(keypoints, json_file, indent=4)
    print(f"Zapisano zaugmentowane keypoints do: {output_path}")

def augment_all_keypoints_in_folder(input_folder, output_folder, max_shift=10,
    ↪ scale_range=(0.8, 1.2)):
    """Augmentacja wszystkich keypoints w folderze i zapisanie wyników"""
    json_files = [f for f in os.listdir(input_folder) if f.endswith('.json')]

    os.makedirs(output_folder, exist_ok=True)
    clear_folder(output_folder)
    for json_file in json_files:
        json_path = os.path.join(input_folder, json_file)

        keypoints = load_keypoints_from_json(json_path)

        augmented_keypoints = augment_keypoints(keypoints, max_shift,
    ↪ scale_range)

        augmented_json_file = f"augmented_{json_file}"
        save_keypoints_to_json(augmented_keypoints, output_folder,
    ↪ augmented_json_file)

```

```

[ ]: # Ścieżki do folderów
input_folder = "/content/keypoints" # Folder z oryginalnymi plikami JSON

```

```
output_folder = "/content/augmented_keypoints" # Folder, gdzie zapiszemy ↵  
↵zaugmentedowane keypoints  
  
augment_all_keypoints_in_folder(input_folder, output_folder, max_shift=10, ↵  
↵scale_range=(0.8, 1.2))
```

Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0024.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0004.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0016.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0022.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0002.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0027.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0018.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0012.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0015.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0011.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0003.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0019.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0026.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0025.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0023.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0008.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0013.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0017.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0010.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0028.json
Zapisano zaugmentedowane keypoints do:

```

/content/augmented_keypoints/augmented_keypoints_0014.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0001.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0009.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0005.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0007.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0021.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0020.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0006.json

```

```

[ ]: import cv2
import glob
import os
from google.colab.patches import cv2_imshow

images_folder = "/content/images" # Folder z obrazami
keypoints_folder = "/content/keypoints" # Folder z plikami JSON z keypoints
augmented_folder = "/content/augmented_keypoints" # Folder z plikami JSON z
↳ augmentedowanymi keypoints

image_files = sorted(glob.glob(os.path.join(images_folder, "*.jpg")))
keypoints_files = sorted(glob.glob(os.path.join(keypoints_folder, "*.json")))
augmented_files = sorted(glob.glob(os.path.join(augmented_folder, "*.json")))

for image_path, keypoints_path, augmented_path in zip(image_files,
↳ keypoints_files, augmented_files):
    image1 = cv2.imread(image_path)
    image2 = cv2.imread(image_path)

    original_loader = KeypointDataLoader(keypoints_path, file_format="json")
    original_keypoints = original_loader.keypoints

    augmented_loader = KeypointDataLoader(augmented_path, file_format="json")
    augmented_keypoints = augmented_loader.keypoints

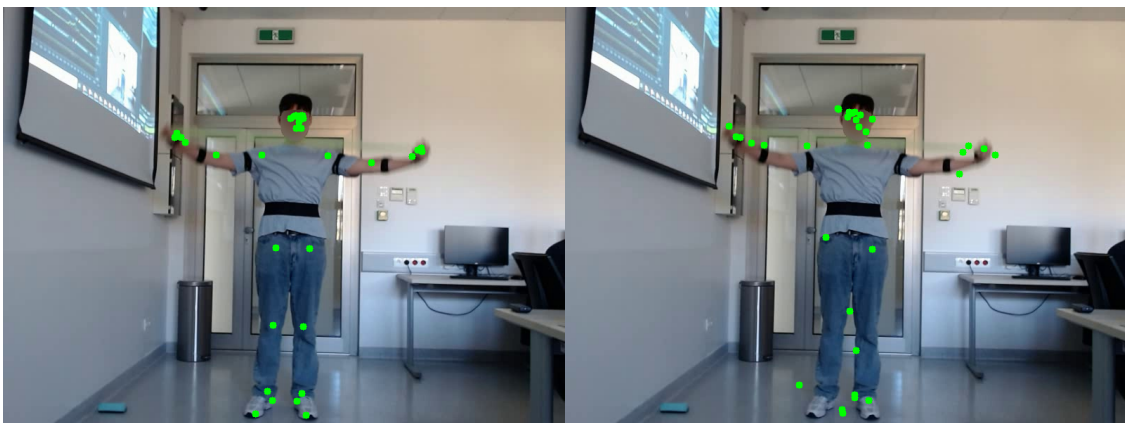
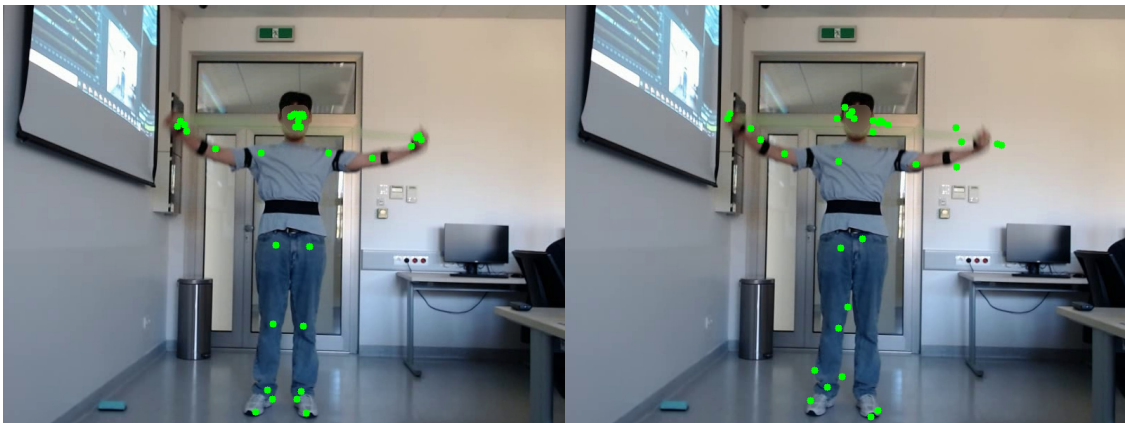
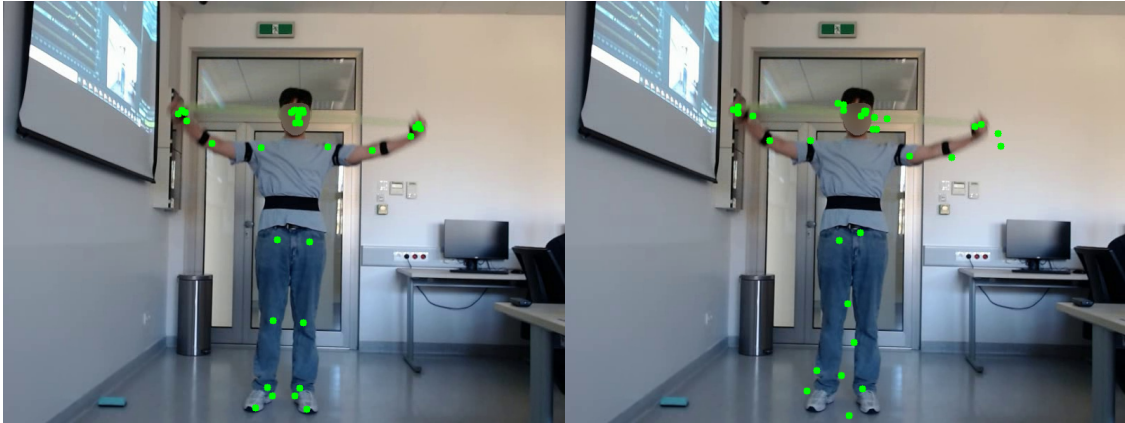
    original_image_with_keypoints = original_loader.
↳ visualize_keypoints_on_image(image1)

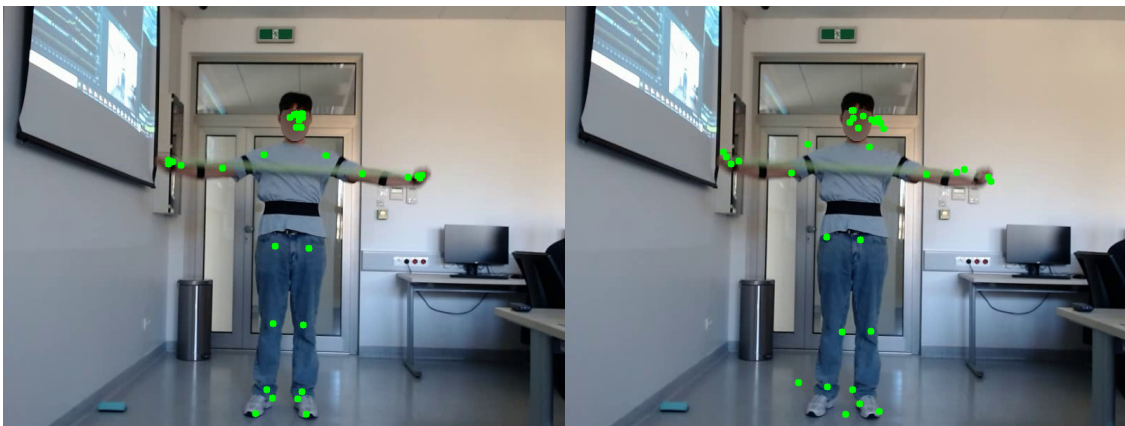
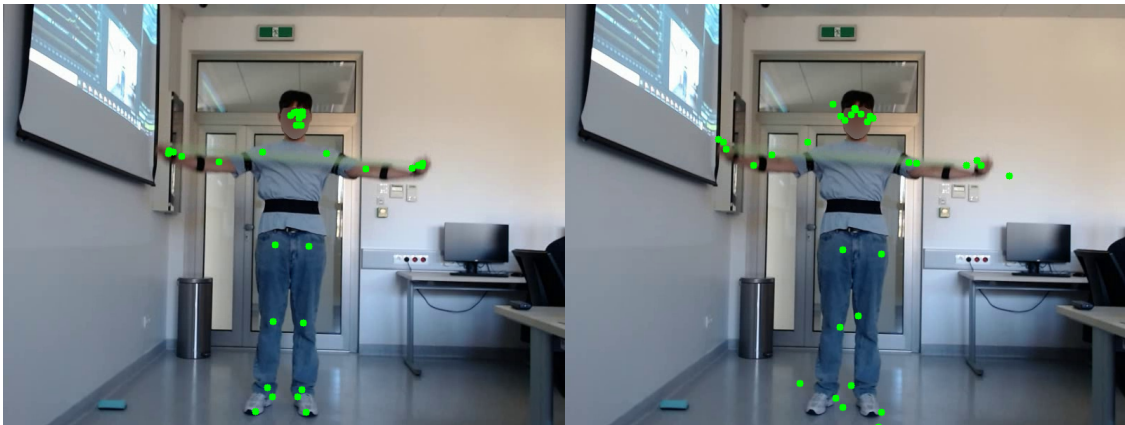
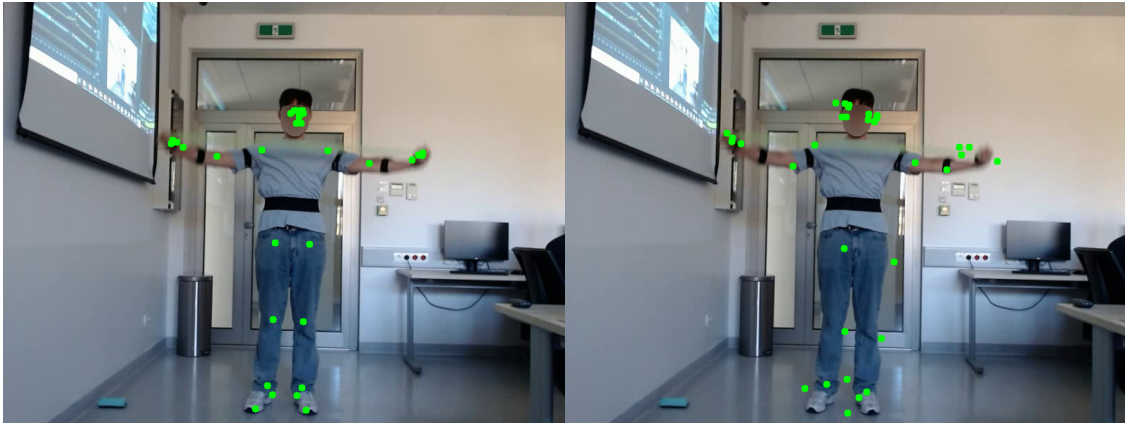
    augmented_loader.keypoints = augmented_keypoints
    augmented_image_with_keypoints = augmented_loader.
↳ visualize_keypoints_on_image(image2)

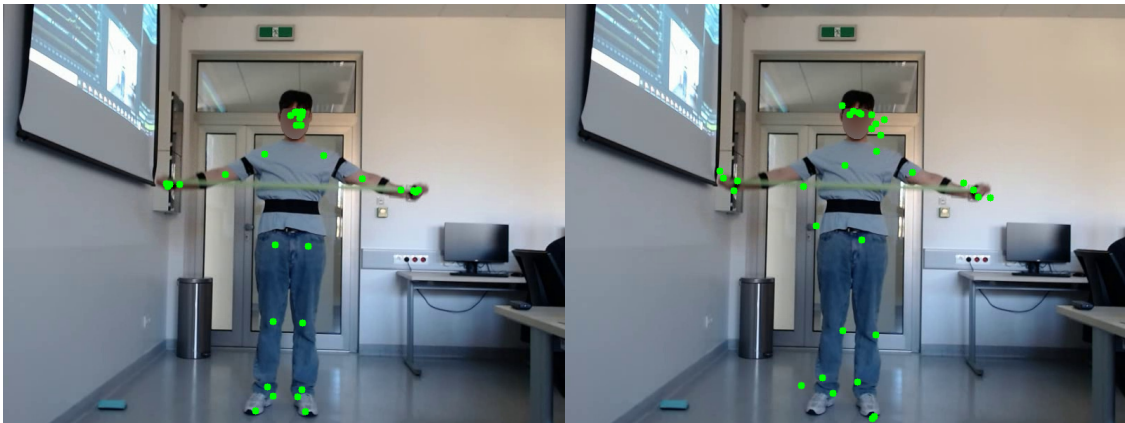
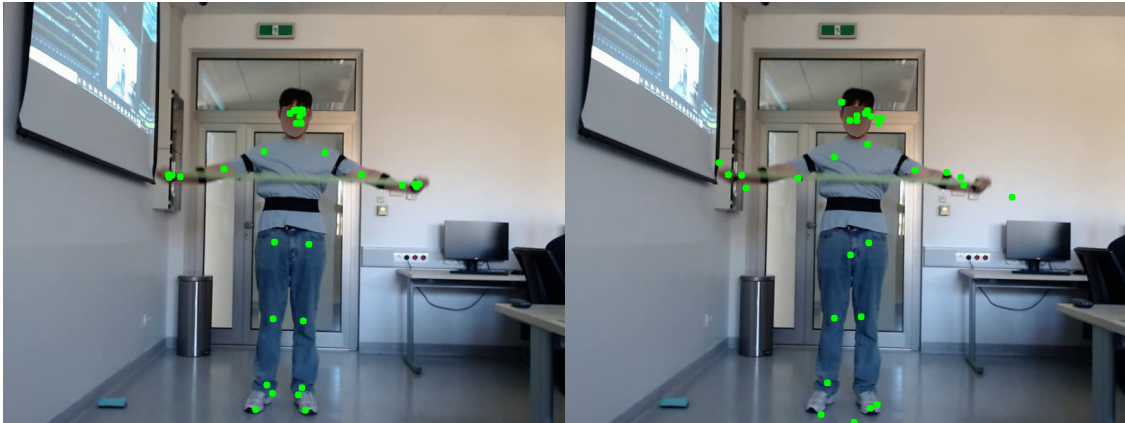
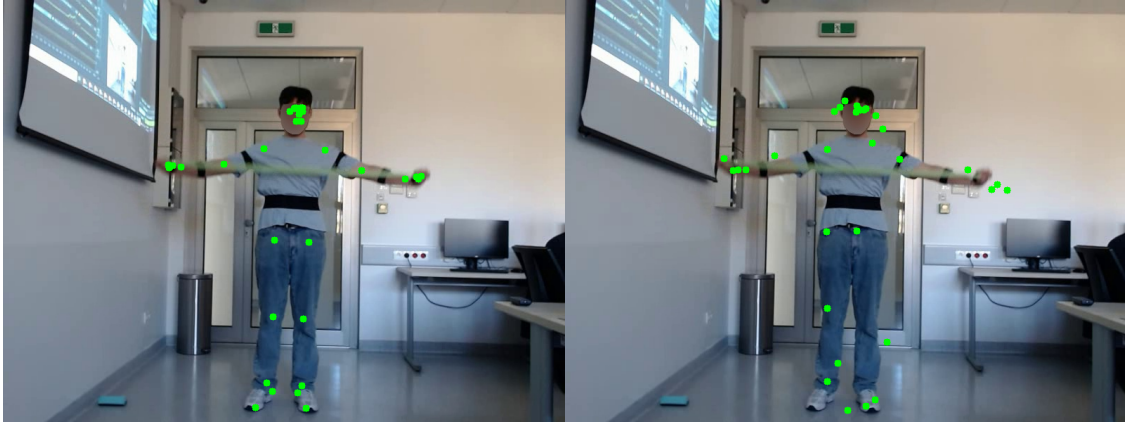
```

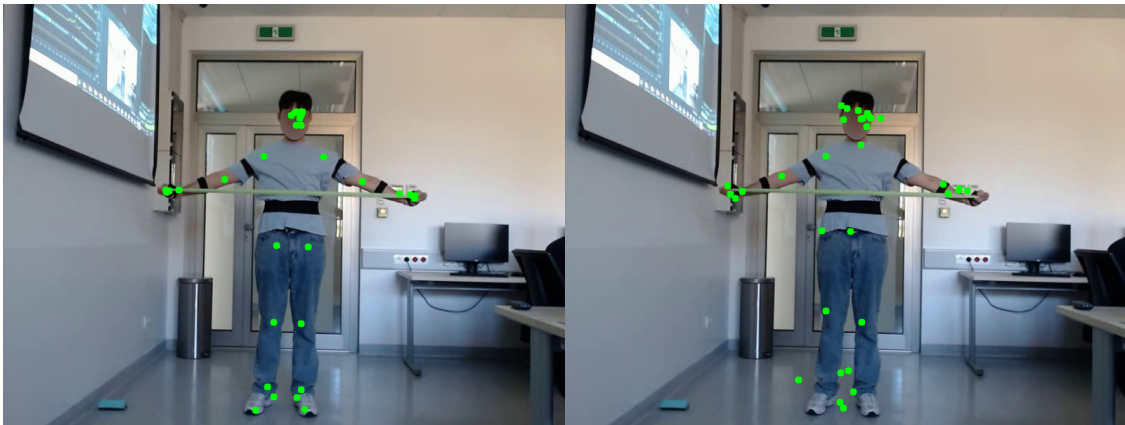
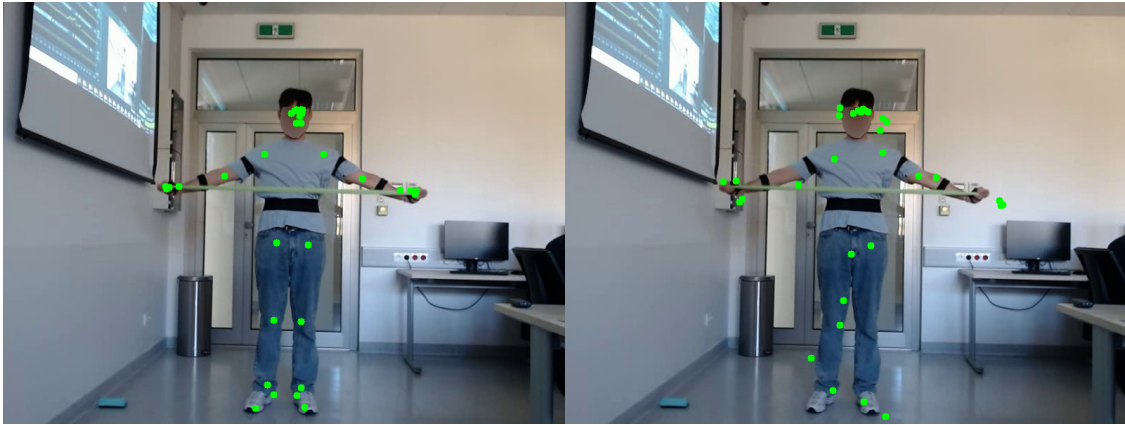
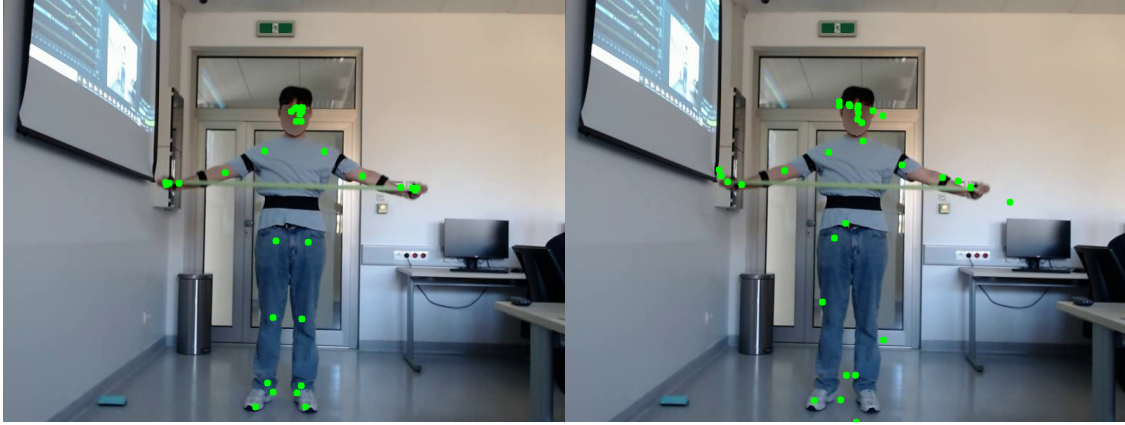


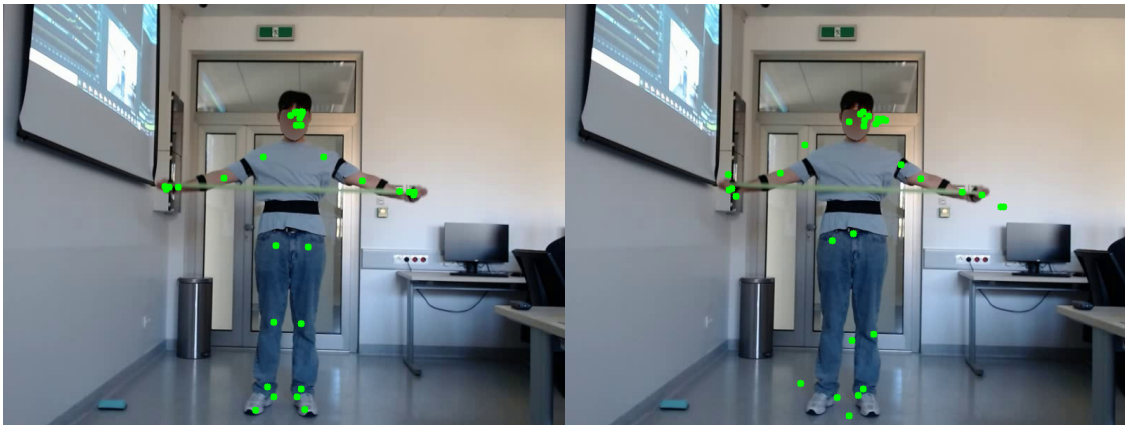
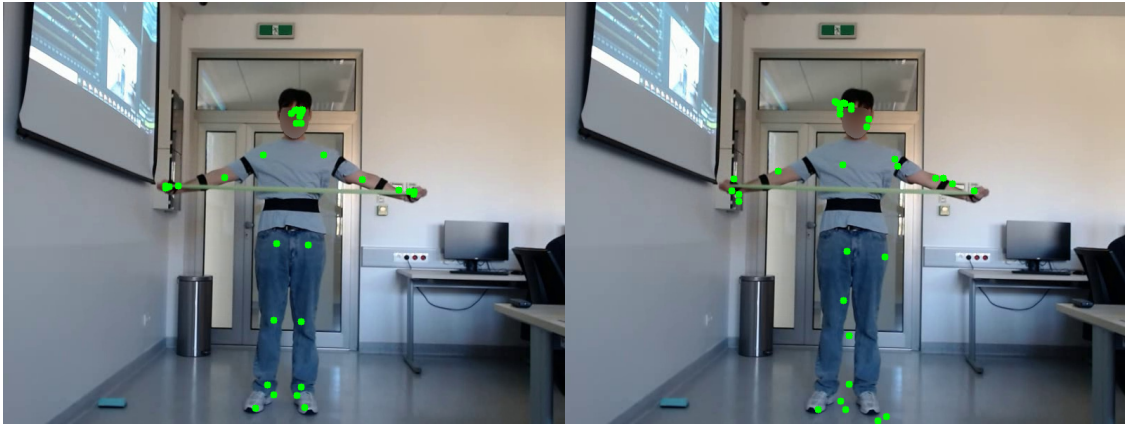
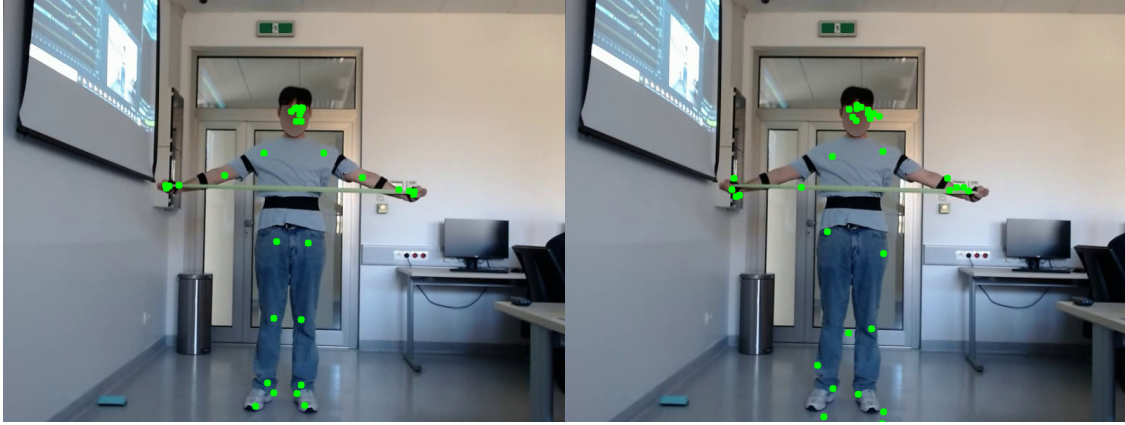
```
combined_image = cv2.hconcat([original_image_with_keypoints,   
↪ augmented_image_with_keypoints])  
  
cv2.imshow('combined_image')
```

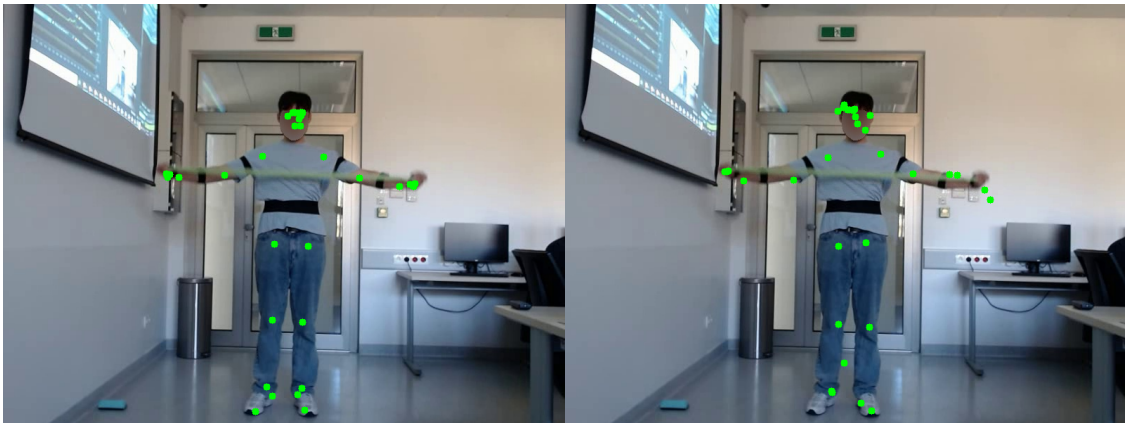
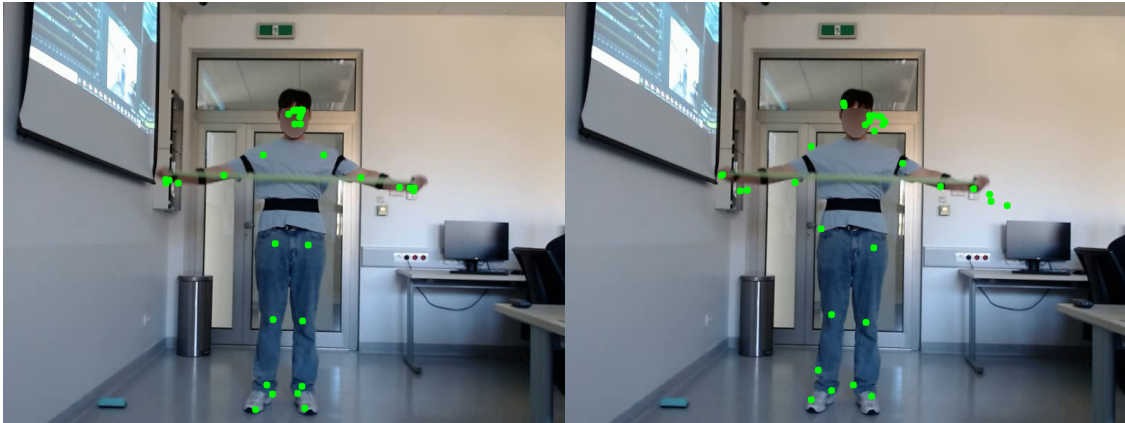
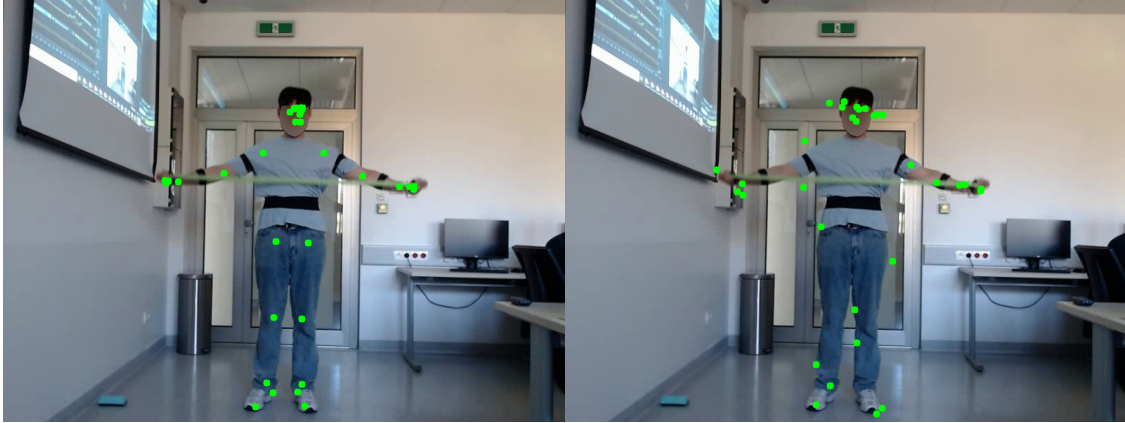


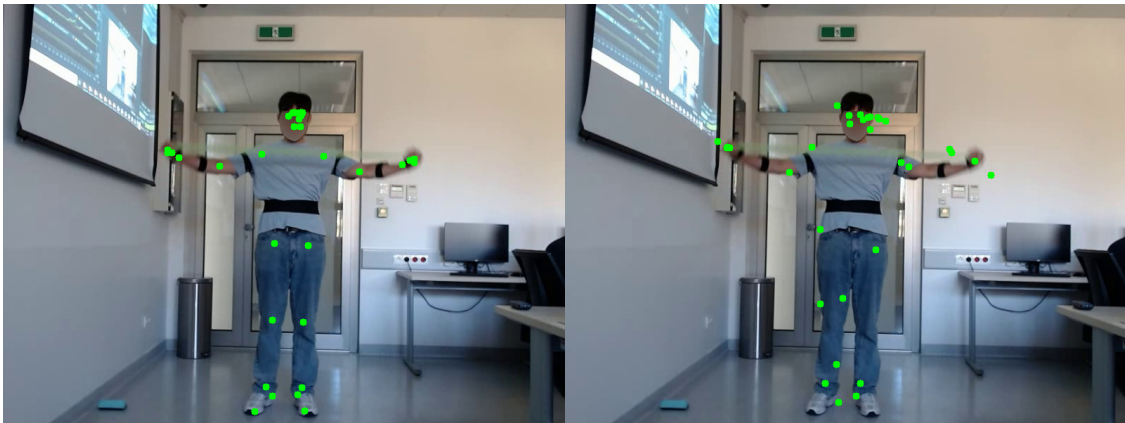
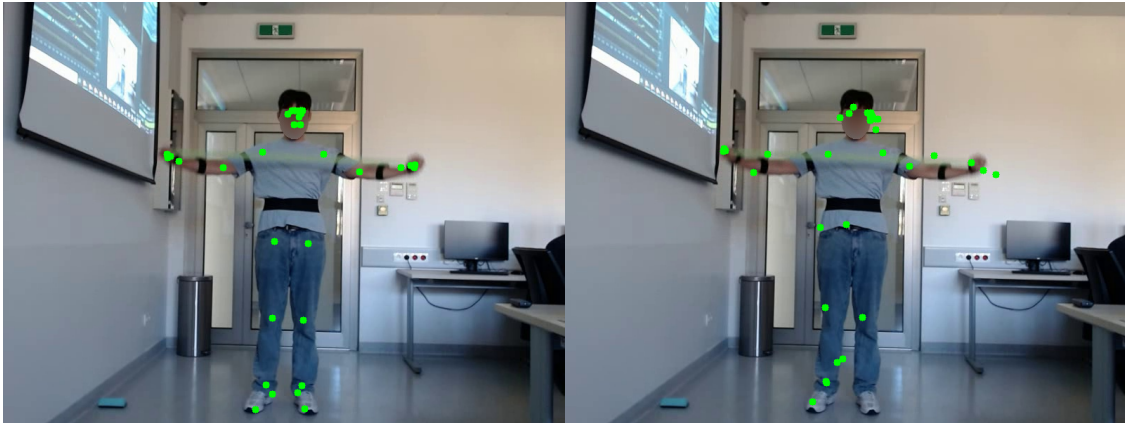
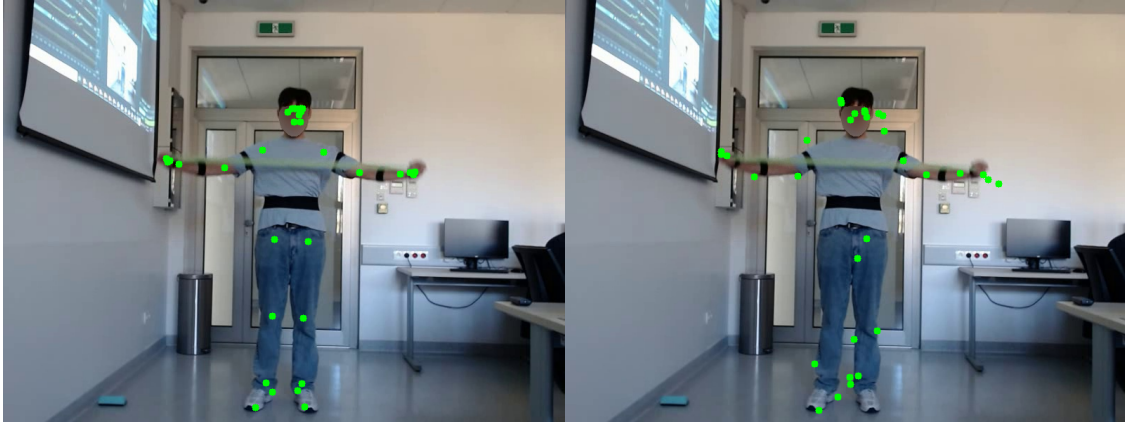


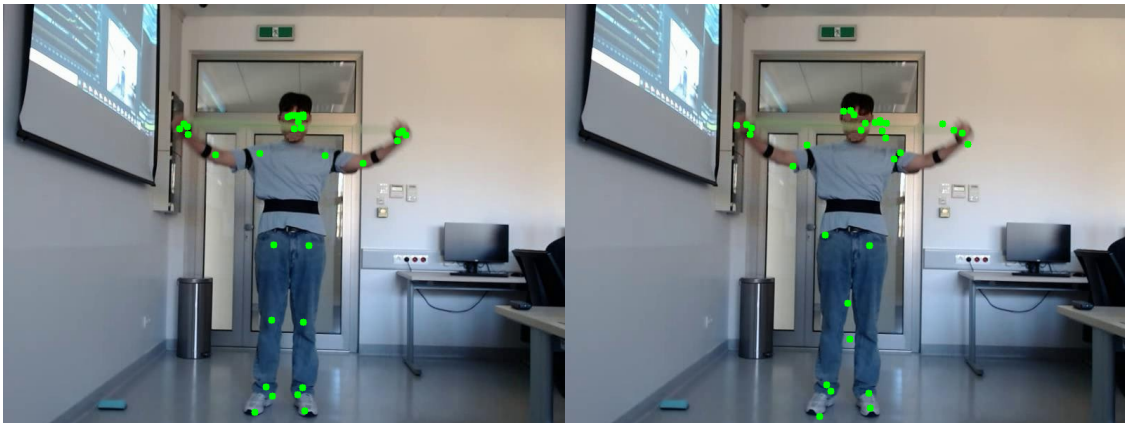
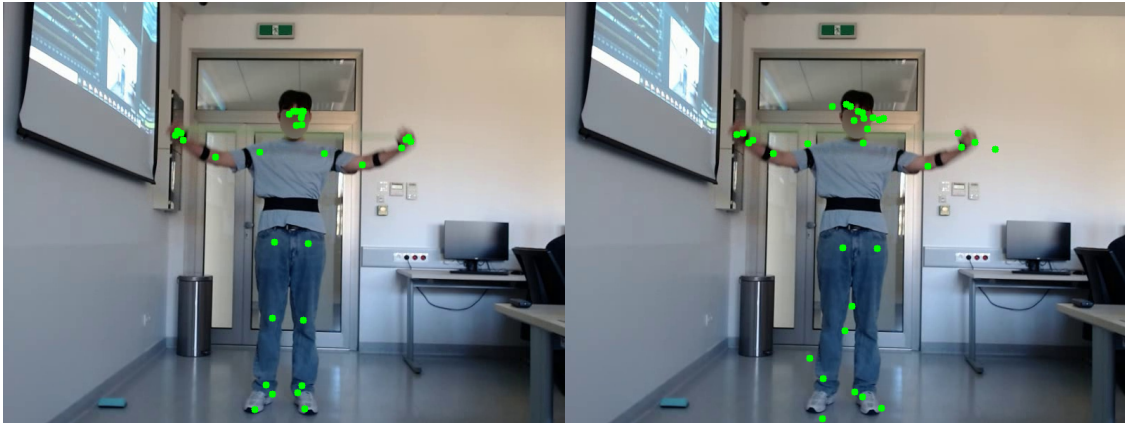
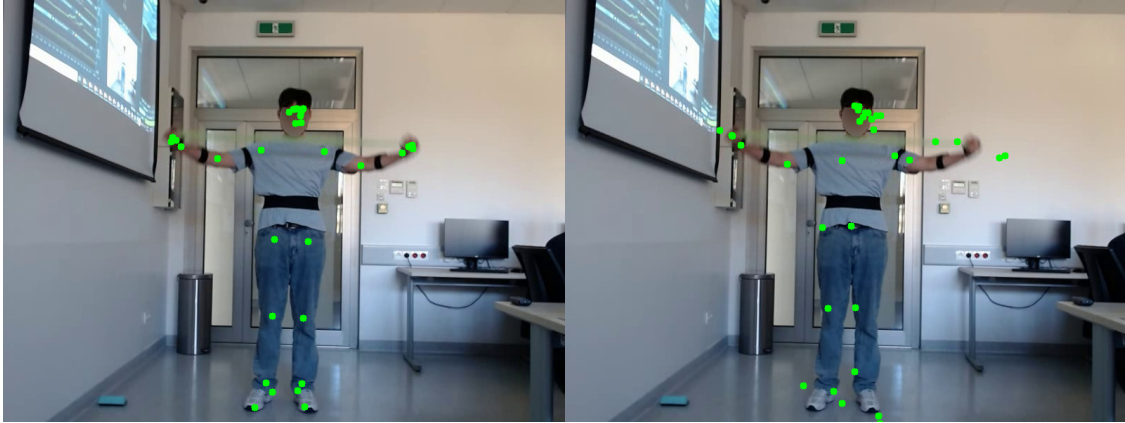


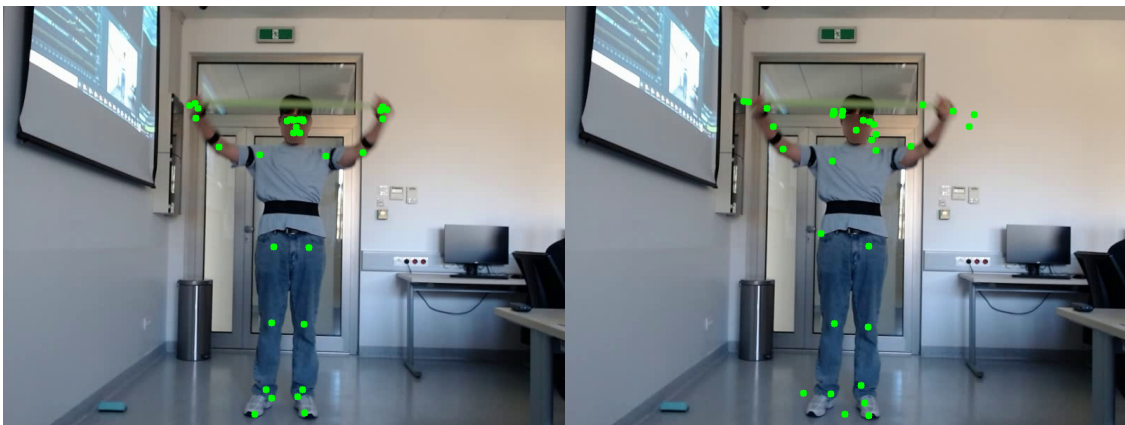
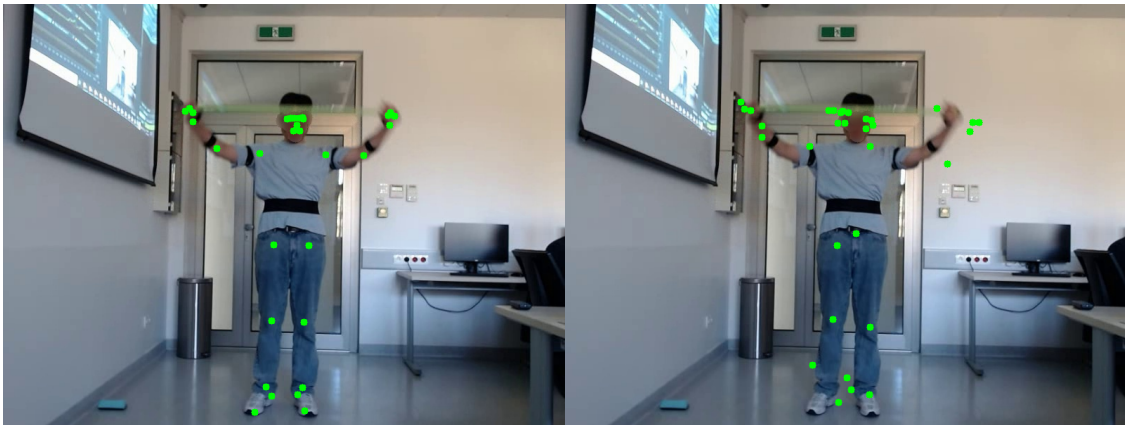
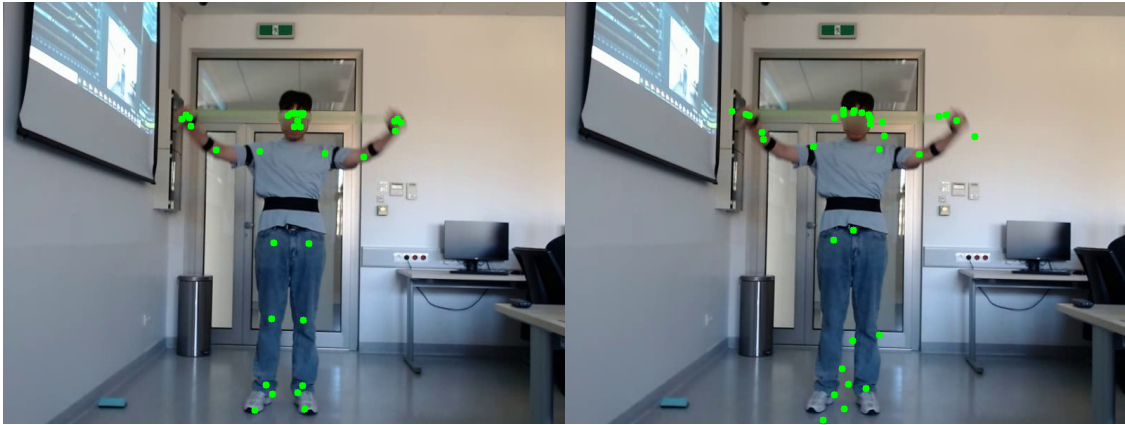


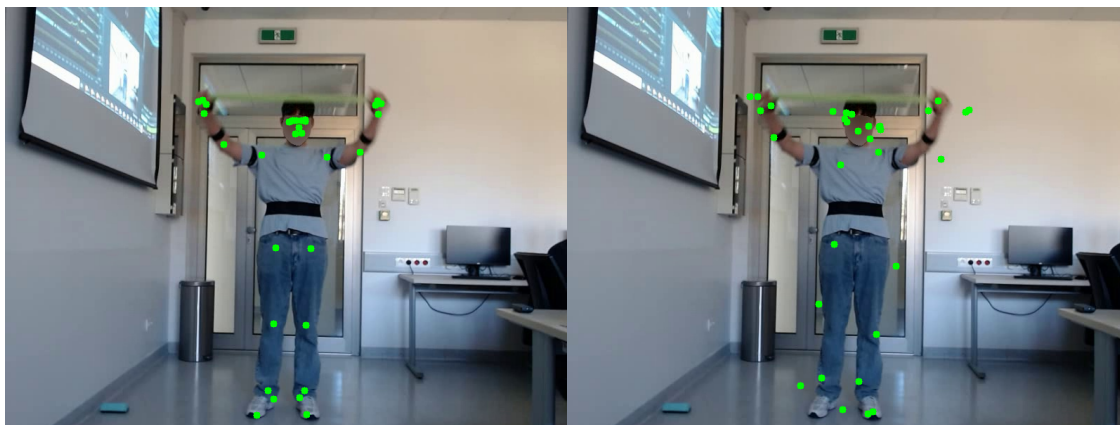












Wynikiem działania kodu jest wizualizacja szkieletu wejściowego oraz po augmentacji danych szkieletowych (w dwóch oknach). Oraz pliki json z keypointami przed oraz po augmentacji. W podanym przykładzie wykorzystano tylko kilkanaście losowych obrazków ponieważ dokument byłby zbyt duży aby wrzucić go na upel.